

COMPILER DESIGN

Handwritten Notes

VIVEKANAND VERNEKAR

GateWallah Parakram C 2024

(Hinglish Weekday)

Other Notes Uploaded Below

<https://t.me/pwgatenotes>

COMPILER DESIGN.

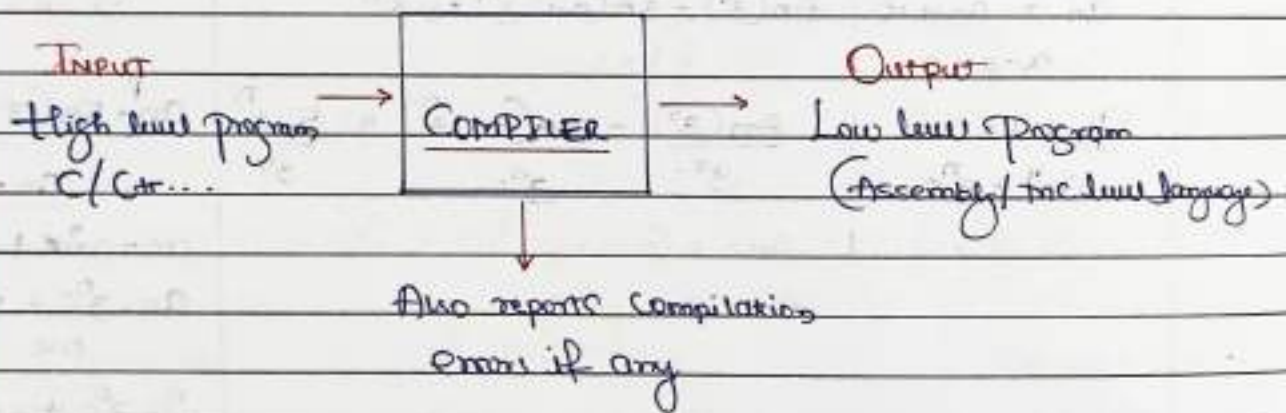
1. Phases of a Compiler
2. Lexical Analysis
3. Syntax Analysis
4. Syntax Directed Translations
5. Intermediate Code Generation
6. Code Optimizations
7. Runtime Environment

Introduction

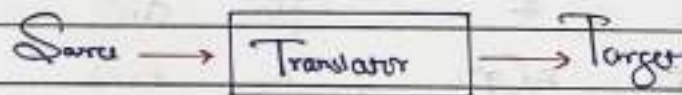
What is Compiler? → Translator

What is use or where can we use Compiler? → Language translation

How Compiler is designed? → 7 phases

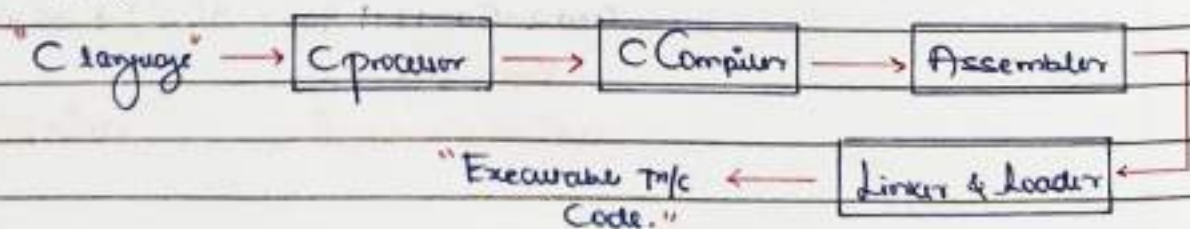


Translator:



Example: Compiler, Interpreter, Editor, Pre-processor, word, linker etc.

Language Translations: (Language to executable code)



LEXICAL ANALYSIS & SYNTAX ANALYSISPre Processor

#define max 10;

int x = max;

void main()

{ int y = max; }

Preprocessor

int x = 10;

void main()

{ int y = 10; }

- #define Constant Exp.

- Replace Constant using Expression.

Assembler:Input R₁, R₂ Assembly Code
R₁ ← R₁, R₂

Assembler

Machine Code.

000110 0110100 011 100

Linker

It resolves all external references

```

=====
Extern int x;
void main()
{
    printf("%d", x);
}

```

Loader

It performs relocation

xxx
address
of
code
block

```

x = x + 10;
address of
variable.

```

} Code block.

Loaders:

- Bootstrap loader
- Swapper
- Other loaders

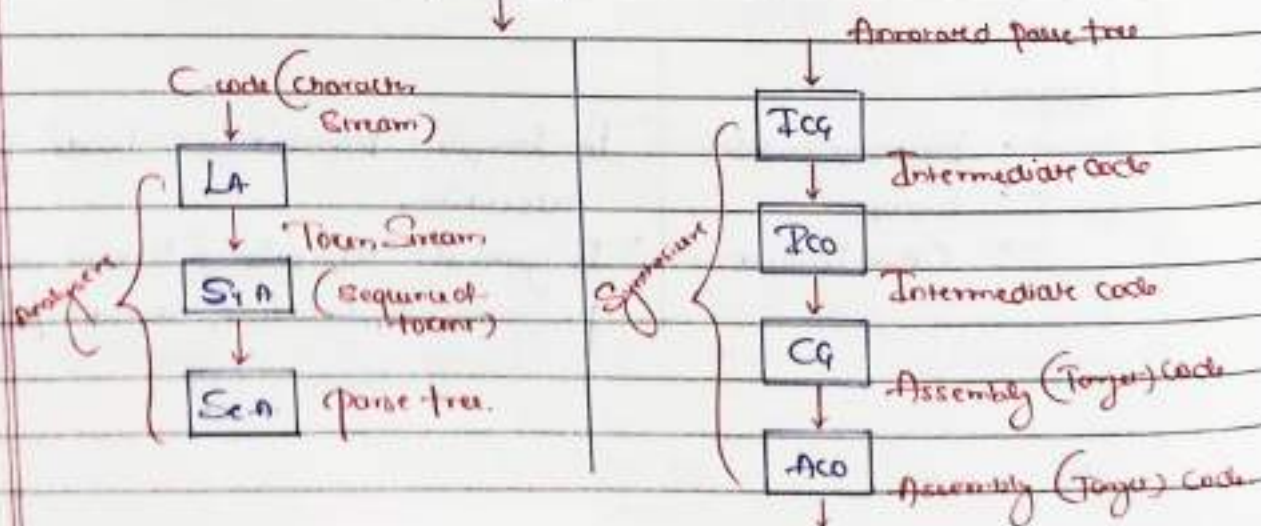
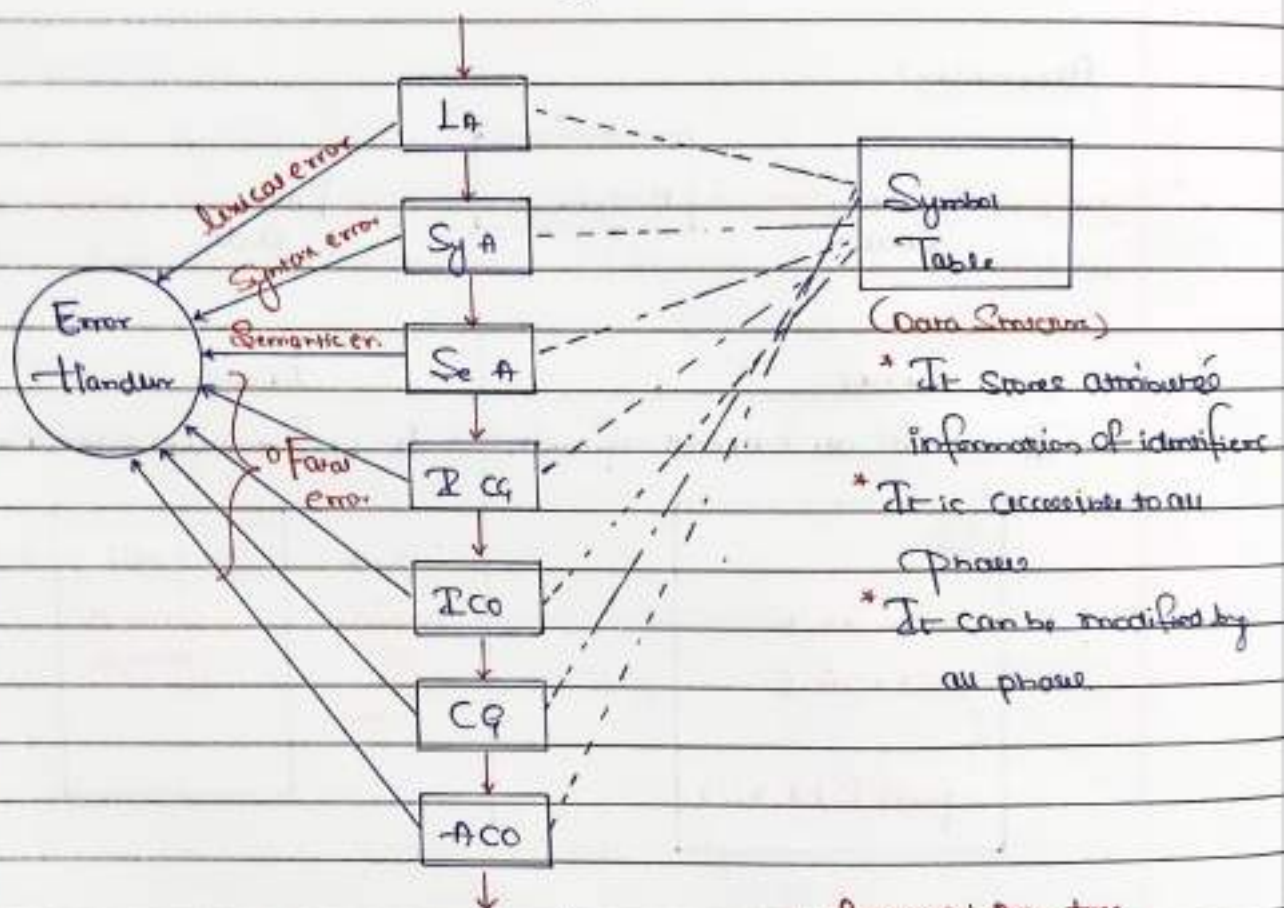
* In language translation: loader performs relocation.

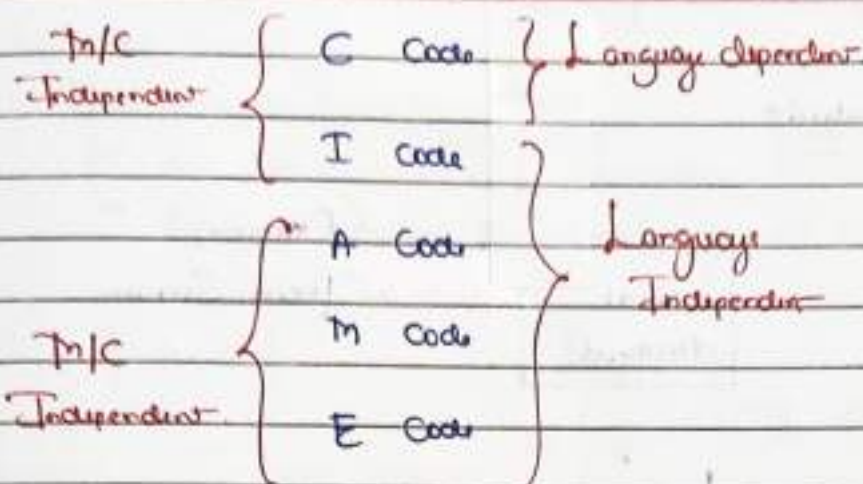
* In general: relocation, loading modules, programs, applications, allocation/deallocation.

Phases of Compiler:

7 phases.

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Intermediate code generator
5. Intermediate code optimization
6. Code generation
7. Assembly (Target) code optimization





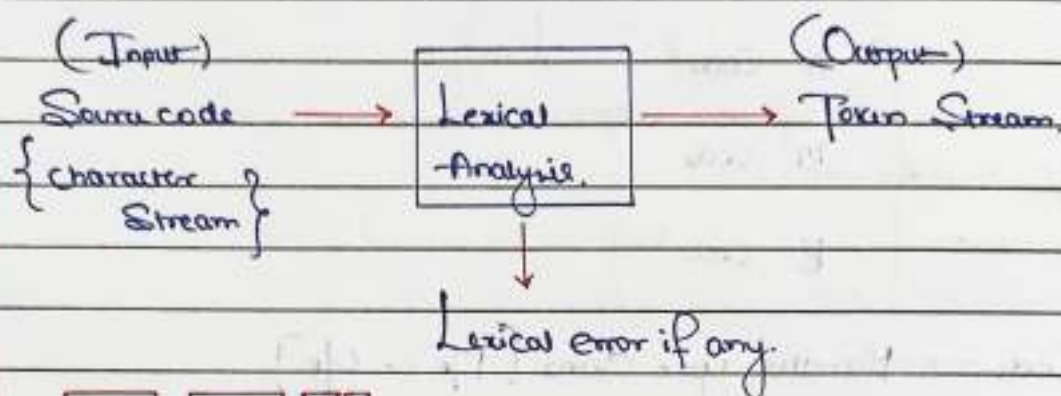
Code $\rightarrow$ Handle your data (I/e or O/e)
 Data $\rightarrow$ Requirement

	C Language	C Compiler
1.	High level code	Machine code (Binary, Executable, low-level)
2.	Machine independent code.	Machine dependent code
3.	Portable $\rightarrow$ Code can run on any machine	Not Portable $\rightarrow$ Code cannot run on any machine $\rightarrow$ runs only on specific m/c.

	Phase	Input	Output	Functionality
1.	LA [A, ii, a]	^A Character Stream	i. Character Stream	a. Token Recognizer
2.	SA [B, iii, b]	^B Token Stream	ii. Token stream	b. Syntax analyzer
3.	SC [C, iv, c]	^C Parse tree	iii. Parse tree	c. Type checker
4.	ICG [D, v, d]	^D Annotated P.T	iv. Annotated P.T.	d. Three address code generator
5.	ICO [E, vi, e]	^E Intermediate code	v. I.C	e. Assembly code generator
6.	CG [F, vii, f]	^F Assembly code	vi. A.C	f. Machine code generator
7.	ACO [G, viii, g]	^G Machine Code	vi. M.C	g. English code generator
		^H English	viii. English	h. Hindi code generator
				i. Intermediate code optimizer
				j. Assembly code optimization

LEXICAL ANALYSIS & SYNTAX ANALYSIS

Lexical Analysis



```

void main()
{ int a; }
  
```

Lexical Analysis :

- Lexical phase
- Lexical Analyser
- First phase of Compiler
- Scanner
- Linker phase
- Token recogniser
- Token generator

Note:

- * Compilation begins from the first character
- * Execution begins from main() function

Lexical Analysis Process

1. It scans whole program character by character
2. It groups character into tokens
3. It ignores extra space and comments
4. It uses "largest/longest prefix rule" (Maximal Munch)
5. It also produces lexical errors if any
6. It creates a symbol table and stores some attribute information of identifiers (variable name, function name... etc)

Symbol Table:

- It is a data structure
- It stores attribute information of identifiers
- It is accessible to all phases of compiler.

eg: void fun (int a)
{ int a; }

Identifiers:

remaining phase

lexeme	Token	line	type	diff..	...
fun	T ₀₁				
x	T ₀₂				
a	T ₀₃				

Lexeme: Smallest unit of (logic) Program

Token: It uniquely represents lexeme.

Type of tokens:

- Keywords
- Literals/Constants
- Operators
- Special Tokens
- Identifiers

Internal representation of lexeme.

Q1. Is "main" keyword?

- No, it is not a keyword
- It is a user defined function
- It is an identifier

- I keyword:
- (1) int, float, double, char, long, short, void, signed, unsigned, volatile, const, auto, static, extern, register.
 - (2) if, else, switch, case, default, while, do, for, break, continue, return, goto.
 - (3) struct, union, enum, typedef.
 - (4) sizeof.

II Operators:

Unary: !, ~, +, -, ++, --, *, &

Arithmetic: +, -, *, /, %

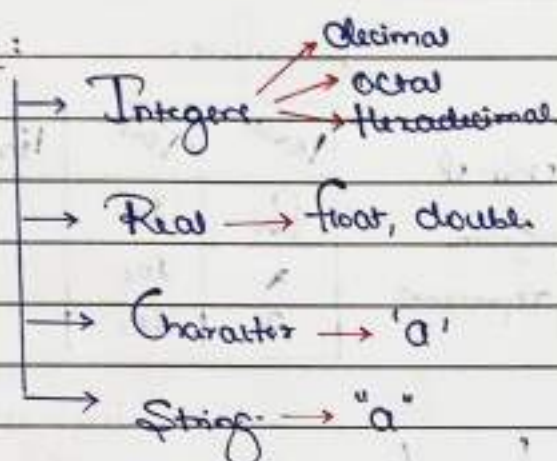
Relational: <, >, <=, >=, ==, !=

Bitwise: &, |, ^, <<, >>

Logical: &&, ||

Assignment: =, +=, -=, /=, %=

- III. Identifier :
- * Name given to variable / function.
 - * Composed of letters, digits, underscore.
 - * Should not start with digit.
 - * Should not be a keyword.

IV. Constants :V. Special Tokens

; , { , } , (,) , [,] , :

Find the tokens in the following codes

1. int x ; 3 ;

(keyword) (identifier) (literal) (separator)

2. int float ; 3 ;

(key) (key) (literal) (separator)

No lexical error but Syntax error.

3. int gate123 ;

4. int 123gate ;

(literal) (literal)

Lexical error.

Invalid Sequence

5. int x = "gate" ;

(keyword) (variable) (string constant) (separator)

6. int x = 0.25 ;

(keyword) (variable) (real constant) (separator)

7. int x = 029 ;

(keyword) (variable) (octal constant)

Lexical error.

8. int x = 0xabc ;

(keyword) (variable) (hexadecimal constant) (separator)

5 tokens

9. int 23 = 45 ;

(keyword) (literal) (literal) (separator)

5 tokens

No lexical error, but syntax error.

10. int /*comment*/ x ;

(keyword) (comment) (variable) (separator)

3 tokens

11. int /*comment*/ x ;

(keyword) (comment) (variable) (separator)

4 tokens

keyword got separated by comment.

12. x = ++y ;

(variable) (operator) (variable) (separator)

5 tokens

13. x = ++y ;

(variable) (operator) (variable) (separator)

5 tokens

14. x ++y ;

(variable) (operator) (variable) (separator)

4 tokens

15. $x = x + 1; y; \Rightarrow 7$

Note:

1. $++ \rightarrow 1$ 2. $++ \rightarrow 2$ 3. $+++ \rightarrow 3$ 4. $+= \rightarrow 1$ 5. $=+ \rightarrow 2$ 6. $== \rightarrow 1$ 7. $! = \rightarrow 1$

No. of tokens

16. $x = ++y; \Rightarrow 6$ 17. $x = y /* comment; \Rightarrow$ lexical error.
begin $\rightarrow$ search for end.18. $x = y /* comment /* comment */; \Rightarrow 4$
begin ignore end19. $x = y comment */; \Rightarrow 7$

No lexical error

20. $x = A[20]; \Rightarrow 7$ 24. $x = y; \Rightarrow 4$ but syntax error21. $x = \text{fun}(2, 3); \Rightarrow 9$ 25. $x = 'a'; \Rightarrow 4$ 22. $x = \text{printf}("gate = \%d", x); \Rightarrow 9$ 26. $x = '\n'; \Rightarrow 4$ 23. $x = \text{scanf}("%d %d", x, y); \Rightarrow 11$ 27. $x = 'abc';$ lexical error. $\rightarrow T_1 = abc$ Input: adabcabeadadabc $T_2 = ad$ Output: $T_2 T_1 T_3 T_2 T_1$ $T_3 = abe$

	T_2	T_1	T_3	T_2	T_2	T_1
	ad	abc	abe	ad	ad	abc
T_1	✓ x	✓ ✓ ✓				
T_2	✓ ✓	✓ x				
T_3	✓ x	✓ ✓ x				

 $\rightarrow T_1 = a^*b$ $T_2 = a^*bc$

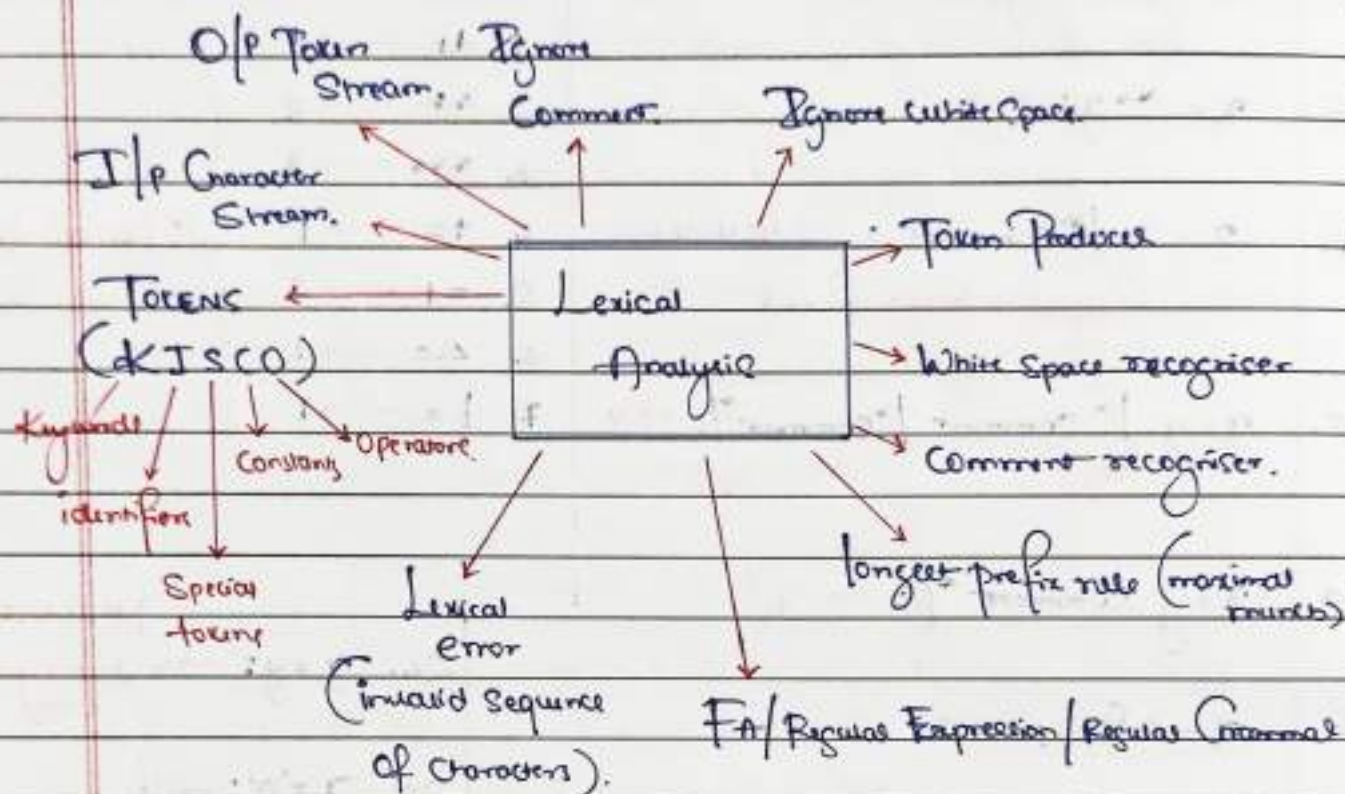
Input: aabcaaaabbbbc

Output: $T_2 T_1 T_1 T_2$

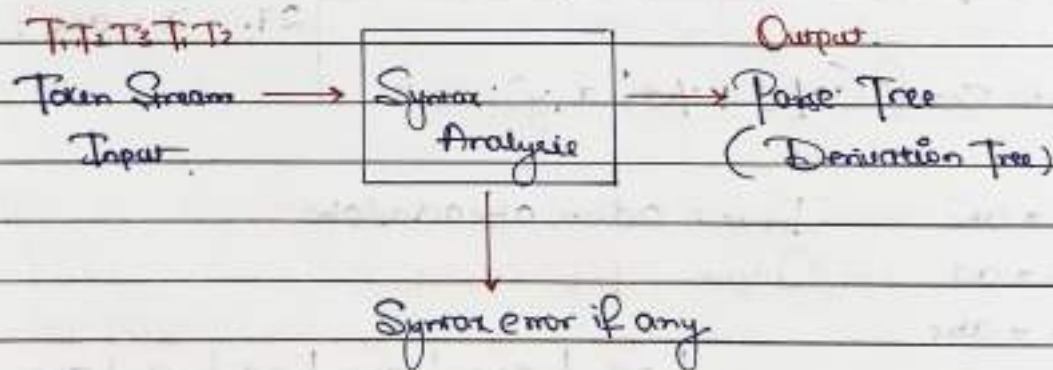
	$T_2 = a^*bc$	$T_1 = a^*b$	$T_1 = a^*b$	$T_2 = a^*bc$
	aabc	aaab	b	bc
T_1	✓ ✓ ✓ x	✓ ✓ ✓ ✓	✓	✓ x
T_2	✓ ✓ ✓ ✓	✓ ✓ ✓ ✓	✓	✓ ✓

Incomplete

a
↓
error
from
'a's



SYNTAX ANALYSIS



Syntax Analysis

- Syntax Analyser
- Parser
- Syntax Verifier
- Parse Tree Generator
- Structure Analyser

Syntax Analysis (Process)

- Syntax error finding
- Context free grammar { Most suitable grammar to represent syntax of programming language }
 - ↳ definition
 - ↳ derivation of string
- Types of CFLs [Ambiguous & Unambiguous]
- Parsers

Find the Syntax Error in the following codes:

1. void main()
{ int x; } → No Error

2. void main()
{ int x, y; } → No Error

3. int x=y; → No Syntax error
but Semantic error.
(x is declared)

4. int x, int y;
Syntax error.

5. int x+y;
Syntax error

6. int y=2;
int x=y;
→ No error

7. int x, y=2; → No error.
x=y=2;

8. int x, y;
x=2, y=2; → No error

9. if (2); → No error.
(True)

10. if (5;
Expression missing.
Syntax error.

11. x, y; No Syntax error
But Semantic error.

12. while (2); No Compilation Error.

13. while ();
Expression missing, Syntax error.

14. for ();
Not correct Syntax, Error.

15. for (; ;);
considered true
No Compilation Error.

16. typedef int x; } x = int.
x y;
int y; valid, No error.
(new name for int alias)

17. for (1, 2, 3); → Syntax error.

18. for (1; 2; 3); → No Compilation Error.

19. for (2, 2, 2); → No Syntax error.
(either Semantic/linear error)

20. for (2; 2; 2);
→ Syntax Error.

Q1) Consider the following ANSI C Program:

```
int main()
{
    Integer e;
    return 0;
}
```

Which one of the following phases of a Compiler will throw an error?

→ Semantic-Analyser

Q2. Consider the following Statements

- I. Symbol-table is accessed only during lexical analysis and syntax analysis.
 - II. Compilers for programming languages that support recursion necessarily need heap storage for memory allocations in the runtime environment. ^{*} (Stack)
 - III. Errors violating the conditions any variable must be declared before its use are detected during syntax analysis.
- Which of them are true?

Ans. ~~None of them are true!~~

Q3. A lexical analyser uses the following patterns to recognise three tokens T_1 , T_2 and T_3 over alphabet $\{a, b, c\}$

$$T_1 : a? (b|c)^* a = (\epsilon + a) (b|c)^* a$$

$$T_2 : b? (a|c)^* b = (\epsilon + b) (a|c)^* b$$

$$T_3 : c? (b|a)^* c = (\epsilon + c) (b|a)^* c$$

Note that ' $x?$ ' means 0 or 1 occurrence of the symbol x . Note also that the analyser outputs the token that matched the longest possible prefix. If the string "bbaacabc" is processed by the analyser, which one of the following is the sequence of tokens it outputs?

Ans. ~~$T_3 T_3$~~

	T_1				T_2		
	b	b	a	a	c	a	b
T_1	✓	✓	✓	×			
T_2	✓	✓	×				
T_3	✓	×	✓	✓	✓	✓	✓

Q4. Match the following:

P. Parse tree

Q. Character Stream

R. Intermediate representation

S. Token Stream

(i) Code generator

(ii) Syntax analyser

(iii) Semantic analyser

(iv) Lexical analyser

Q5. Match the following:

- | | |
|------------------------|---------------------------|
| P. Lexical analysis | (i) Leftmost derivation |
| Q. Top down parsing | (ii) type checking |
| R. Semantic analysis | (iii) regular expressions |
| S. Runtime environment | (iv) activation records |

Q6. In a compiler, keywords of a language are recognised during
 Ans. The lexical analysis of a program.

Q7. Which data structure in a compiler is used for managing information about variables and their attributes?

Ans. Symbol Table.

Context Free	Context Sensitive
{Structure, Syntax}	{Semantic} {Meaning}
$S + V \neq 0$ ✓ I am Student ✓ I are Student	I am Student (Semantically correct) I are Student (Semantically wrong)
Integer x; Syntax correct	Integer x;; Semantically wrong

Context Free Grammar [CFG]

It can be used to represent Syntax/Structure of programming languages.

$$CFG = (V, T, P, S)$$

Each rule in P:

LHS $\rightarrow$ RHS

V $\rightarrow$ Any sequence

V $(V \cup T)^*$

- $\rightarrow$ Start Symbol $S \in V$
- $\rightarrow$ Set of rule (production)
(production rule)
- $\rightarrow$ Set of terminals
- $\rightarrow$ Set of Variable (non-terminals)

Identify Cases..

- | | | | |
|---|----------------------------|---|--|
| 1 | $S \rightarrow E$ | 4 | $S \rightarrow aSb \mid Ss \mid a$ ✓ |
| 2 | $S \rightarrow Sab \mid a$ | 5 | $S \rightarrow S \mid a \mid \epsilon$ ✓ |
| 3 | $S \rightarrow AB \mid a$ | 6 | $S \rightarrow aSb$ |
- $AB \rightarrow a$ $aS \rightarrow bb$

 $S = \text{program.}$

$\text{program} \Rightarrow R I () \{ D ; \}$
 $R \rightarrow \text{void / int}$
 $I \rightarrow \text{identifier}$
 $D \rightarrow T ;$
 $T \rightarrow \text{int / float / char}$

```

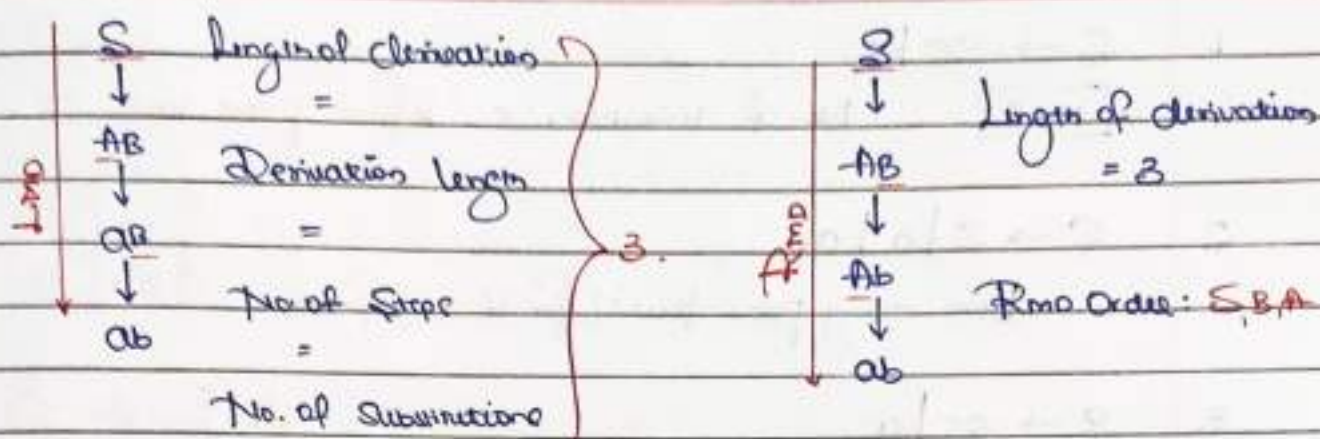
void main()
{
    int x;
}

```

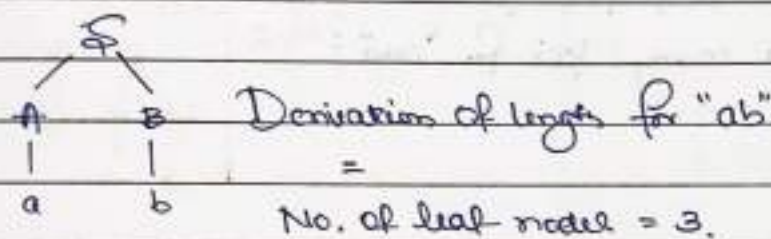
 $V = \{ \text{program, R, I, D, T} \}$ $T = \{ (,), \{, \}, \text{void, int, Identifier, float, char} \}$ Derivation of a String:

- Linear {
 I Leftmost derivation (LMD)
 II Rightmost derivation (RMD)
 Non-linear {
 III Parse tree derivation (PT)

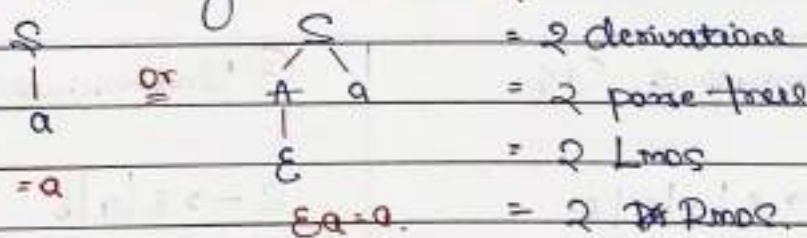
	LMD	RMD	PT
$S \rightarrow AB$ $A \rightarrow a$ $B \rightarrow b$ $w = ab$ String	S $\downarrow$ AB $\downarrow$ aB $\downarrow$ ab In every step left most non-terminal is substituted.	S $\downarrow$ AB $\downarrow$ Ab $\downarrow$ ab Rightmost non-term is substituted.	S $\swarrow \searrow$ $A \quad B$ $\downarrow \quad \downarrow$ $a \quad b$ Leaf: - terminal or ϵ Nonleaf: non-terminal

Lmo Order $\Rightarrow$ S, A, B

Note: Lmo and Rmo need not be same.



eg: $S \rightarrow a \mid Aa$ $\Rightarrow$ $S \rightarrow a$
 $A \rightarrow \epsilon \mid b$ $\Rightarrow$ $S \rightarrow Aa$
 $A \rightarrow \epsilon, A \rightarrow b$

 $w=a \Rightarrow$ How many derivations for w ?

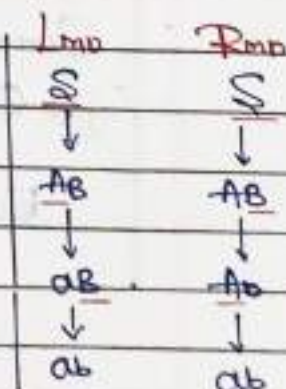
"For a given string all these will be same."

eg: $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

In this example:
 Lmo and Rmo are different.

No. of Lmo for ab
 = No. of Rmo for ab = 1.

No. of Steps in Lmo
 =
 No. of Steps in Rmo
 = 3



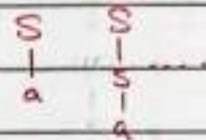
1. $S \rightarrow SS | \epsilon$

$w = \epsilon$

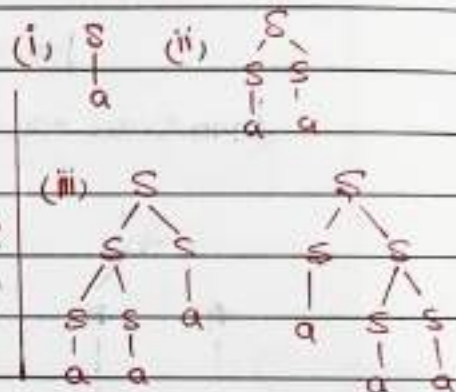
No. of derivations = Infinite parse tree

2. $S \rightarrow S | a | ab$

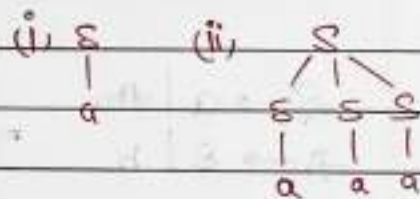
How many parse trees for 'a' : Infinite



3. $S \rightarrow SS | a$

(i) How many Pts for 'a'? $\rightarrow 1$ (ii) How many Pts for 'aa'? $\rightarrow 1$ (iii) How many Pts for 'aaa'? $\rightarrow 2$ 

4. $S \rightarrow SSS | a$

(i) How many Pts for 'a'? $\rightarrow 1$ (ii) How many Pts for 'aa'? $\rightarrow 0$ (iii) How many Pts for 'aaa'? $\rightarrow 1$ Types of CFGs based on derivations

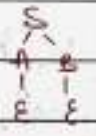
① Ambiguous CFG	② Unambiguous CFG
$S \rightarrow \epsilon a b A$ $A \rightarrow a$	$S \rightarrow \epsilon a b$
There is some string, which has more than one derivation. Some string ≥ 1 PT $\exists w \in L(CFG) \rightarrow \geq 1$ PT.	$\epsilon = 1$ pt $a = 1$ pt $b = 1$ pt. Every string only 1 PT $\forall w \in L(CFG) \rightarrow 1$ PT.
* Easy to write	* Better

Identify Ambiguous and Unambiguous CFGs

1. $S \rightarrow a$: Unambiguous ✓ $a \Rightarrow 1 \text{ PT}$

2. $S \rightarrow a | \epsilon$: Unambiguous ✓ $\epsilon = 1, a = 1$

3. $S \rightarrow AB | \epsilon$: Unambiguous ✓ $\epsilon = 1 \text{ PT}$
 $A \rightarrow \epsilon, B \rightarrow a$ $a = 1 \text{ PT}$

4. $S \rightarrow AB$
 $A \rightarrow \epsilon, B \rightarrow \epsilon$
: Unambiguous ✓


7. $S \rightarrow AB$ $\epsilon = 1 \text{ PT}$
 $A \rightarrow aA | \epsilon$ $a = 2 \text{ PT}$
 $B \rightarrow aB | \epsilon$
: Ambiguous ✓

5. $S \rightarrow AB$ $\epsilon = 1 \text{ PT}$
 $A \rightarrow a | \epsilon$ $a = b = 1 \text{ PT}$
 $B \rightarrow b | \epsilon$ $ab = 1 \text{ PT}$
: Unambiguous ✓

8. $S \rightarrow SS | \epsilon$
: Ambiguous ✓

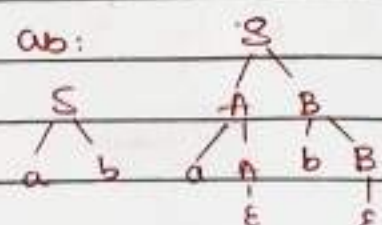
6. $S \rightarrow AB$ $\epsilon = 1 \text{ PT}$
 $A \rightarrow aA | \epsilon$ $a = b = 1 \text{ PT}$
 $B \rightarrow bB | \epsilon$: Unambiguous ✓

9. $S \rightarrow SS | a$
 $a = 1 \text{ PT}$
 $aa = 1 \text{ PT}$: Ambiguous ✓
 $aaa = 2 \text{ PT}$

10. $S \rightarrow Sas | b$: left and right rec using same S.
Ambiguous ✓

11. $S \rightarrow Sa | bS | c$: Ambiguous ✓

12. $E \rightarrow E + E | a$: Ambiguous ✓

13. $S \rightarrow AB | ab$ ab :
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | \epsilon$
: Ambiguous ✓


Elimination of left Recursion

(Conversion from left recursion to right recursion)

* Top-down Parser follows LRP

$$S \rightarrow \underbrace{Sa}_\text{first symbol} | \underbrace{b}_\text{first symbol}$$

$$\boxed{b | \dots\dots}$$

first symbol

① $S \rightarrow \underbrace{Sa}_\text{left recursion} | \underbrace{b}_\text{non-left recursion}$

$$\boxed{S \rightarrow bs'}$$

$$\boxed{s' \rightarrow as' | \epsilon}$$

$$\boxed{S \rightarrow bx}$$

$$\boxed{x \rightarrow ax | \epsilon}$$

$$A \rightarrow A\alpha | \beta$$

$$L = \{ b, ba, baa, baab, \dots \}$$

$$= \{ ba^0, ba^1, ba^2, \dots \}$$

$$= ba^*$$

$$A \rightarrow BA'$$

$$A' \rightarrow \alpha A' | \epsilon$$

Formula:
(Direct left recursion elimination)

$$A \rightarrow \underbrace{A\alpha_1 | A\alpha_2 | \dots | A\alpha_n}_\text{left recursion} | \underbrace{B | B_2 | \dots | B_k}_\text{non-left recursion}$$

$$A \rightarrow BA' | B_2A' | \dots | B_kA'$$

$$A' \rightarrow \underbrace{\alpha_1 A' | \alpha_2 A' | \dots | \alpha_n A'}_\text{Right Recursion} | \epsilon$$

$\alpha \in (V \cup \epsilon)^*$
 $B \in (V \cup \epsilon)^*$
 Should not start with A'

② $S \rightarrow \underbrace{Sa}_\alpha | \underbrace{cb}_\beta | \underbrace{d}_\gamma$

$$S \rightarrow Sx | B_1 | B_2$$

$$\Rightarrow \boxed{S \rightarrow cs' | ds'}$$

$$\boxed{s' \rightarrow abs' | \epsilon}$$

or

$$\boxed{S \rightarrow cs' | ds' | c' | d}$$

$$\boxed{s' \rightarrow abs' | ab}$$

3. $S \rightarrow \underbrace{Sa}_\alpha | \underbrace{Sb}_\alpha | \underbrace{cd}_\alpha | \underbrace{e}_\beta | \underbrace{fe}_\beta | \underbrace{sg}_\beta$

$$S \rightarrow \underbrace{Sa}_\alpha | \underbrace{Sb}_\alpha | \underbrace{Sg}_\alpha | \underbrace{cd}_\beta | \underbrace{e}_\beta | \underbrace{fe}_\beta$$

Left Recursion Right Recursion

$$\boxed{S \rightarrow cdx | ex | fsx}$$

$$\boxed{x \rightarrow ax | bx | gx | \epsilon}$$

Direct left recursion Elimination

Left Recursion

$$S \rightarrow Sa | b$$

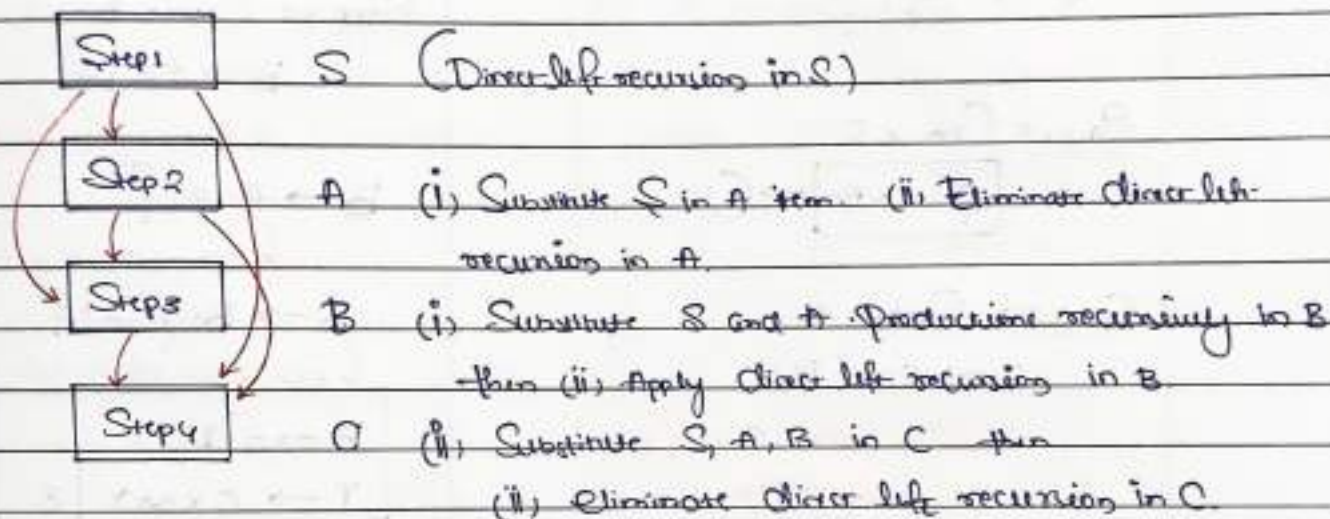
* Direct left recursion

$$S \rightarrow Aa | d$$

$$A \rightarrow Sb | c$$

* Indirect Left recursion

* We can convert left recursion into direct left recursion using "Substitution".



1. $S \rightarrow Aa | Sb | c$
 $A \rightarrow Sb | Ae | f$

Step 1: Choose S productions:

$$S \rightarrow Sb | Aa | c$$

left rec non left recursion

Direct left recursion

$S \rightarrow Aax cx$	$\rightarrow \textcircled{1}$
$x \rightarrow bx \epsilon$	

Step 2: Choose A productions:

$$A \rightarrow Sb | Ae | f$$

Substitute

S rules if appears in 1st position.

$$S \rightarrow Aax | cx$$

$$A \rightarrow \underbrace{Aaxb}_{\alpha_1} | \underbrace{cxb}_{\beta_1} | \underbrace{Ae}_{\alpha_2} | \underbrace{f}_{\beta_2}$$

Direct left recursion

$A \rightarrow Cxb fy$	$\rightarrow \textcircled{2}$
$y \rightarrow axby ey \epsilon$	

2. $E \rightarrow E + T | F$

$T \rightarrow F * T | F$

$F \rightarrow id$

Step 1: $E \rightarrow E+1 \mid 1$

$$\begin{cases} E \rightarrow Fx \\ x \rightarrow 1Tx \mid \epsilon \end{cases} \quad (1)$$

Step 2: $T \rightarrow F+T \mid F$ — (2)

Let E be 1st partition, so no substitution,
no direct left recursion.

Step 3: $F \rightarrow id$ — (3)

{ Ans: 1, 2, 3 combined }

3
 $S \rightarrow aA$
 $A \rightarrow Sb \mid Bc$
 $B \rightarrow Sa \mid f$

Step 1: Choose S

$$S \rightarrow aA \quad (1)$$

Step 2: Choose A

$$A \rightarrow Sb \mid Bc$$

Substitute 'S' in 1st place
 $S \rightarrow aA$

$$A \rightarrow aAb \mid Bc$$

eliminate direct left recursion in A.

$$\begin{cases} A \rightarrow BCx \\ x \rightarrow abx \mid \epsilon \end{cases} \quad (2)$$

Step 3: Choose B

$$B \rightarrow Sa \mid f$$

Substitute 'S' in B

$$B \rightarrow aAa \mid f$$

Substitute 'A' in B.

$$B \rightarrow Bcxa \mid f$$

eliminate direct left recursion.

$$\begin{cases} B \rightarrow fY \\ Y \rightarrow BcxaY \mid \epsilon \end{cases} \quad (3)$$

$$X \rightarrow X\alpha_1 \mid X\alpha_2 \mid B_1 \mid B_2 \mid P_3$$

$$X \rightarrow B_1x' \mid B_2x' \mid P_3x'$$

$$x' \rightarrow \alpha_1x' \mid \alpha_2x' \mid \epsilon$$

Left Refactoring:

one left factor
 $S \rightarrow ab \mid acd$
 One begin common prefix
 issue is backtracking

CFG

Elimination of common
 prefix

(left factoring)

Left factored CFG

(CFG without common
 prefix).

$$\begin{cases} S \rightarrow ax \\ x \rightarrow b \mid cd \end{cases}$$

1. $S \rightarrow a$: left factored CFG2. $S \rightarrow a | \epsilon | bc$: left factored CFG3. $S \rightarrow aA$
 $A \rightarrow b$ } left factored CFG4. $S \rightarrow ab | a | cd$

$$\begin{array}{|l} S \rightarrow ax | cd \\ X \rightarrow b | \epsilon \end{array}$$
left factored
5. $S \rightarrow a | ab | abc | ef$

$$S \rightarrow ax | ef \rightarrow (1)$$

$$X \rightarrow \epsilon | b | bc$$

$$\begin{array}{|l} X \rightarrow \epsilon | bY \\ Y \rightarrow \epsilon | c \end{array} \rightarrow (2)$$
6. $S \rightarrow Ab | aca | d$

$$S \rightarrow acb | gbl | aca | d$$

$$\begin{array}{l} \text{Ans: } S \rightarrow ax | gb | d \\ X \rightarrow cb | ca \\ A \rightarrow ae | g \end{array}$$
FIRST SET $S \rightarrow abc | de | \epsilon$ $\text{First}(S) = \{a, d, \epsilon\}$
 $= \{x | x \text{ is either } \epsilon \text{ or terminal where terminal is derived as 1st symbol from } S\}$

First(x) = It is set of terminals (may include ϵ) where each terminal is derived as 1st symbol from x.

 $\text{First}(abc) = \{a\}$ $\text{First}(de) = \{d\}$

We need x rule for computing $\text{First}(x)$

FOLLOW SET $X \rightarrow axb | Yb | b$ $Y \rightarrow bxc | Xc | \epsilon$ $\text{Follow}(x) = \{b, e, m\}$
 $= \{t | t \text{ is a terminal derived as 1st symbol after } x\}$

Follow(x) = It is set of terminals where every terminal is derived as 1st symbol after x.

We need where CFG where production contain x helps to compute $\text{Follow}(x)$.

Note: If S is start symbol - then $\text{Follow}(S)$ should contain $\$$.
"\$" is special end-terminal

eg: $S \rightarrow Sa | Sb | c$
 $\text{Follow}(S) = \{ \$, a, b \}$
 (Start) (rules)

(1) $S \rightarrow a | \epsilon | bc$ $\text{Follow}(S) = \{ \$ \}$
 $\text{First}(S) = \{ a, \epsilon, b \}$

(2) $S \rightarrow Sa | \epsilon$ $\text{Follow}(S) = \{ \$, a \}$
 $\text{First}(S) = \{ \epsilon, a \}$

(3) $S \rightarrow Ab | cde$ $\text{Follow}(S) = \{ \$ \}$ $\text{First}(S) = \{ c, a, b, \epsilon \}$
 $A \rightarrow ae | \epsilon | fg$ $\text{Follow}(A) = \{ b \}$ $\text{First}(A) = \{ a, \epsilon, f \}$

(4) $S \rightarrow A$ $\text{First}(S) = \{ \epsilon \}$ $\text{Follow}(S) = \{ \$ \}$
 $A \rightarrow \epsilon$ $\text{First}(A) = \{ \epsilon \}$ $\text{Follow}(A) = \{ \$ \}$

Note: $X \rightarrow \alpha Y$

$\text{Follow}(Y) = \text{Follow}(X) \cup \{ \text{LHS} \}$

Xa
 $\alpha Y a$ (Same as $\text{Follow}(X)$)

..... Xa
 $\downarrow$
 $\alpha Y a$
 $\downarrow$

If you want to find after Y , it is same as whatever after X .

First(X):

$X \rightarrow \bigcirc | \bigcirc | \bigcirc$
 (1st symbol) (1st) (1st)
 (terminal)

Follow(X): where CFG take rule which contains X in RHS
 $\text{LHS} \rightarrow \boxed{X}$

5. $S \rightarrow AB$

	S	A	B
$A \rightarrow a$ First	{a}	{a}	{b}
$B \rightarrow b$ Follow	{ ϕ }	{b}	{ ϕ }

6. $S \rightarrow AaBb$

	S	A	B
$A \rightarrow f$ First	{f}	{f}	{f}
$B \rightarrow f$ Follow	{ ϕ }	{a}	{b}

7. $S \rightarrow AB$

	S	A	B
$A \rightarrow ab$ First	{f}	{f}	{f}
$B \rightarrow \epsilon$ Follow	{ ϕ }	{a}	{b}

8. $S \rightarrow AB$

	S	A	B
$A \rightarrow \epsilon$ First	{e}	{e}	{e}
$B \rightarrow ef$ Follow	{ ϕ }	{e}	{ ϕ }

9. $S \rightarrow Ss|a$

	S
First	{a}
Follow	{ ϕ , a}

10. $S \rightarrow Ss|(s)|\epsilon$

	S
First	{ ϵ , c}
Follow	{ ϕ , c, s}

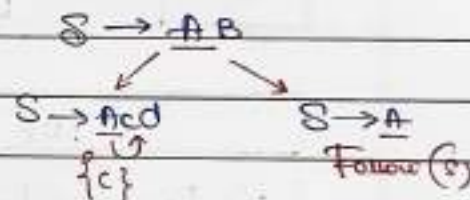
11. $E \rightarrow E + E | E * E | (E) | a$

First(E) = {c, a} = 2

Follow(E) = {+, *,), ϕ } = 4

12. $S \rightarrow AB$

	S	A	B
$A \rightarrow ab \epsilon$ First	{a, c, ϵ }	{a, ϵ }	{c, ϵ }
$B \rightarrow cd \epsilon$ Follow	{ ϕ }	{c, ϕ }	{ ϕ }



13. $S \rightarrow ABC$

	S	A	B	C
$A \rightarrow ab \epsilon$ First	{a, c, f, ϵ }	{a, ϵ }	{c, ϵ }	{f, ϵ }
$B \rightarrow cd \epsilon$ Follow	{ ϕ }	{c, f, ϕ }	{f, ϕ }	{ ϕ }
$C \rightarrow fg \epsilon$				

14. $E \rightarrow E + T | T$

	E	T	F
$T \rightarrow F * T F$ First	{c, id}	{c, id}	{c, id}
$F \rightarrow (E) id$ Follow	{ ϕ , +,)}	{ ϕ , +,)}	{*, ϕ , +,)}

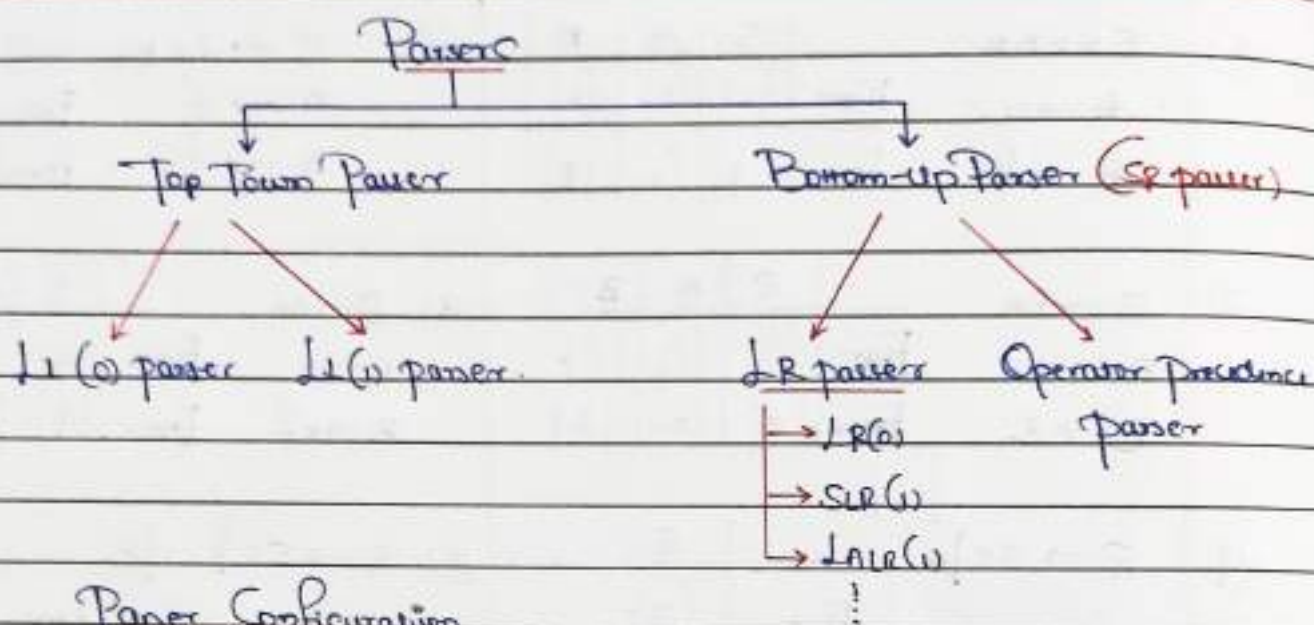
$Fo(E) = \{*\} \cup Fo(T)$

$T \rightarrow F * T \quad T \rightarrow E$

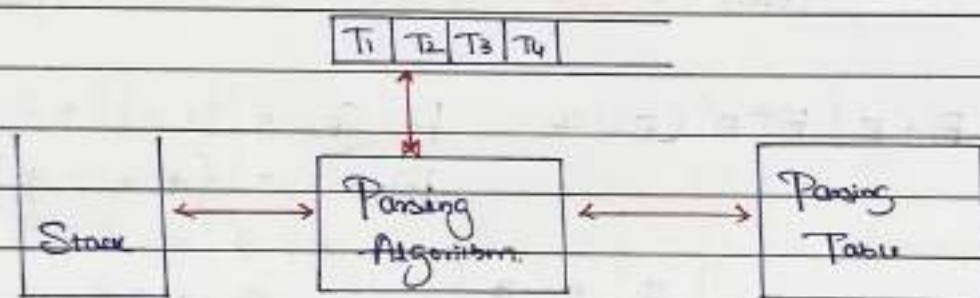
$Follow(E) = \{*\}, \quad Follow(F) = Follow(T)$

$= \{*\} \cup Follow(T)$

is always present
in Follow (star symbol)



Parser Configuration



Top	Bottom-up
I) Followed "Lma" (L)	II) Followed "Rmi" in reverse (R)
$\downarrow$ LL(1) left to right scan	$\downarrow$ LR left to right scan
II) LL(1) parser def - aut.	II) LR(1) parser powerful

LL(1) Parser:

* Top down parser

* Predictive parser

* It can be implemented in 2 ways:

✓ (i) Recursive-descent parser [procedure driven approach]

$S \rightarrow aA$ $s()$

{ if (nextToken == a) $A()$ }

✓ Every production rule has procedure.

$S \rightarrow aA$	a
S	$S \rightarrow aA$

If CFG is unambiguous, non-left recursive, and left factored
- then need not be LL(1)

Q2 Which of the following is necessary to be LL(1) CFG?

- A. Ambiguous, left recursive, left factored CFG
- B. Unambiguous, non left recursive, non left factored CFG
- ☒ C. Unambiguous, non left recursive, left factored CFG
- D. None of these

If CFG is LL(1) then it is unambiguous, non left recursive and left factored.

LL(1) Table Construction

Step 1: Compute first set for every non-terminal in CFG.

Step 2: If any first set contains ϵ then compute follow set for the same non-terminal.

Step 3: Fill the table using First and Follow sets

<u>First</u> $m[s, a]$:		a
S		Fill with all productions of S which derive 'a' as 1 st symbol

<u>Follow</u> $m[s, b]$:		b
S		Fill all productions of S which derives ϵ

① $S \rightarrow aSb \mid \epsilon$

$\text{First}(S) = \{a, \epsilon\}$

$\text{Follow}(S) = \{\$, b\}$

Every entry of table has at most 1 Rule

→ Given CFG is LL(1)

m	a	b	\$
S	$S \rightarrow aSb$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

② $S \rightarrow aSa \mid \epsilon$

$\text{First}(S) = \{a, \epsilon\}$

$\text{Follow}(S) = \{a, \$ \}$

This CFG is unambiguous,
non left recursive, left factored.

$M[s, a]$ has more

than 1 rule

Given CFG is
not LL(1)

	a	\$
S	$S \rightarrow asa$ $S \rightarrow \epsilon$	$S \rightarrow \epsilon$

③ $S \rightarrow SS \mid (s) \mid \epsilon$

$\text{First}(S) = \{ (, \epsilon \}$

$\text{Follow}(S) = \{), (, \$ \}$

$S \rightarrow SS$
 ϵ
 $S \rightarrow \epsilon$

only focus on deriving ϵ

	(	)	\$
S	$S \rightarrow (S)$ $S \rightarrow \epsilon$	$S \rightarrow SS$ $S \rightarrow \epsilon$	$S \rightarrow SS$ $S \rightarrow \epsilon$

④ $S \rightarrow AB$

$F_1(S) = \{ a, b, \epsilon \}$

$F_0(S) = \{ \$ \}$

$A \rightarrow Aa \mid \epsilon$

$F_1(A) = \{ a, \epsilon \}$

$F_0(A) = \{ b, \$, a \}$

$B \rightarrow Bb \mid \epsilon$

$F_1(B) = \{ b, \epsilon \}$

$F_0(B) = \{ b, \$ \}$

	a	b	\$
S	AE	AB	AB
A	AAE	ϵ	ϵ
B		BbE	ϵ

⑤ $S \rightarrow aAb$

$F_1(S) = \{ a \}$

$F_0(A) = \{ b \}$

$A \rightarrow Bb \mid \epsilon$

$F_1(A) = \{ a, \epsilon \}$

$B \rightarrow ab$

$F_1(B) = \{ a \}$

	a	b	\$
S	$S \rightarrow aAb$		
A	$A \rightarrow Bb$	$A \rightarrow \epsilon$	
B	$B \rightarrow ab$		

Shortcut for identifying LL(1) CFG

Rule 1: $X \rightarrow \alpha_1 \mid \alpha_2$

If $\text{first}(\alpha_1) \cap \text{first}(\alpha_2) \neq \emptyset$ then given CFG is not LL(1)

$S \rightarrow a \mid ab \mid c \rightarrow \text{It is not LL(1)}$
 Rule 1
 Rule 1
 Rule 1
 $= \emptyset$

Rule 2: $X \rightarrow \alpha \mid \epsilon$

If $\text{first}(\alpha) \cap \text{follow}(X) \neq \emptyset$, then given CFG is not LL(1)
 If it is not

eg. $S \rightarrow aSb \mid \epsilon$

$\text{First}(a^nb^n) \cap \text{Follow}(S) = \phi$
 $\{a\} \cap \{\$, b\} = \phi$ } It is LL(1)

① $S \rightarrow a \mid b \mid cd \mid \epsilon \Rightarrow$ It is LL(1)

② $S \rightarrow Sa \mid bs \mid c \mid d \mid \epsilon \Rightarrow$ Not LL(1)

③ $S \rightarrow a \mid bc \mid acf \Rightarrow$ Not LL(1)

④ $S \rightarrow AB \mid ab$
 $A \rightarrow a \mid \epsilon$
 $B \rightarrow b \mid \epsilon$ $\Rightarrow$ Not LL(1)

⑤ $S \rightarrow aAbB \mid bAaB \mid \epsilon$
 $A \rightarrow S$
 $B \rightarrow S$ $\Rightarrow$ Not LL(1)

Q1. If CFG is not LL(1) then which is possible?

- ☒ A. Ambiguous
- ☒ B. Left recursive
- ☒ C. Non left factored
- ☒ D. Some table entry may contain more than one production.

Q2. If CFG is not LL(1) then which is True?

- ☐ A. Ambiguous
 - ☐ B. Left recursive
 - ☐ C. Non left factored
 - ☒ D. Some table entry may contain more than one production.
- guaranteed

Q3. If CFG is LL(1) then which is True?

- ☒ A. Unambiguous
- ☒ B. Non left recursive
- ☒ C. Left factored
- ☒ D. A, B, & C
- ☒ E. Every entry of table has at most 1 production

Q4. CFG is LL(1) if CFG is

A. Unambiguous

B. Non left recursive

C. Left factored

D. A, B & C

~~E.~~ Every entry of table has atleast 1 production

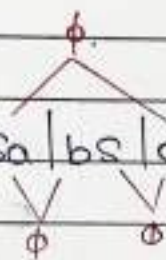
~~F.~~ A, B, C, & E

G. B & C

~~H.~~ C & E

~~I.~~ B & E

~~J.~~ A, B, E.



① $S \rightarrow aSa \mid bS \mid c$ is LL(1)

② $S \rightarrow aAbB \mid bAaB \mid \epsilon$

$A \rightarrow S$

$B \rightarrow S$

	a	b	\$
S	I, E ₁	II, E ₂	III
A			
B			

What is E₁ and E₂?

$\text{First}(S) = \{a, b, \epsilon\}$

$\text{Follow}(S) = \{\$, a, b\}$

$E_1 = \text{I, III}$

$= \{S \rightarrow aAbB, S \rightarrow \epsilon\}$

$E_2 = \text{II, III} = \{S \rightarrow bAaB, S \rightarrow \epsilon\}$

③ $P \rightarrow zQRS$

$Q \rightarrow yz \mid z$

$R \rightarrow w \mid \epsilon$

$S \rightarrow y$

$\text{Follow}(Q) = \{w, y\}$

④ $E \rightarrow E-T \mid T$

$T \rightarrow T+F \mid F$

$F \rightarrow (E) \mid \text{id}$

Equivalent non-left recursive grammar is:

$E \rightarrow Tx$

$x \rightarrow -Tx \mid \epsilon$

$T \rightarrow Fy$

$y \rightarrow +Fy \mid \epsilon$

$F \rightarrow (E) \mid \text{id}$

(5)

$S \rightarrow daT \mid Rf$		a	b	c	d	f	\$
$T \rightarrow aS \mid baT \mid \epsilon$	S			①		②	
$R \rightarrow caTR \mid \epsilon$	T			②			④
	R						

$$Fo(S) = Fo(T) \cup \{f\}$$

$$Fo(R) = \{f\} \cup \cancel{Fo(R)}$$

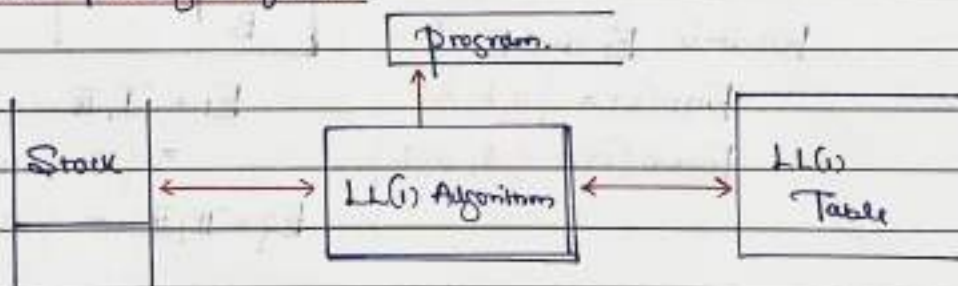
$$First(S) = \{a, c, f\}$$

$$First(T) = \{a, b, \epsilon\}$$

$$Follow(T) = \{f, c, f\}$$

$S \rightarrow daT$	$T \rightarrow baT$	$R \rightarrow caTR$	$R \rightarrow ca$	TR
$Fo(S) = Fo(S)$ $= \cancel{Fo(R)}$ $\cup \{f\}$	X	$\{c\} \cup Fo(R)$ $\{c\} \cup \{f\}$		$\{c\} \cup Fo(R)$

LL(1) Parsing Algorithm



I/p = $ab\epsilon$

Given Table:

M	a	b	\$
S	$S \rightarrow aSb$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

I Prediction

If $T.O.S = \text{Non-terminal}$ then

$M[T.O.S, \text{input}] : LHS \rightarrow RHS$

(i) Pop T.O.S (LHS)

(ii) Push RHS in reverse order

II Match T.O.S == I/p

(i) Pop T.O.S

(ii) Document pointer.

Step 1:

S
\$

$ab\epsilon$ (Prediction)

$M[s, a], S \rightarrow aSb \rightarrow$ (i) pop s

(ii) push b, S, a

Step 2:

a
s
b
\$

$\uparrow$ $ab\$$
 $T.O.S == i/p.$
 $\therefore$ match
 (i) pop s
 (ii) increment pointer.

Step 3:

s
b
\$

$\uparrow$ $b\$$
 $M[s, b] \quad \$ \rightarrow \epsilon$
 (i) pop s
 (ii) push nothing.

Step 4:

b
\$

$\uparrow$ $b\$$
 $T.O.S == i/p.$
 (i) pop b
 (ii) increment pointer.

Step 5:

\$

$\uparrow$ $\$$
 $T.O.S == \$ == i/p$
 - ACCEPT ✓
 (No syntax error)

eg. $S \rightarrow asb | \epsilon$

ab	S		Prediction	a	match		Prediction	
	$\downarrow$		$\rightarrow$	S	$\rightarrow$	S	$\rightarrow$	
Lmo	asb	S	$S \rightarrow asb$	b		b		b
	$\downarrow$	\$		\$		\$		\$
	ab							
								$\downarrow$ match

What is max size of stack by assuming initially $\$$ and S are already on stack?
 $\Rightarrow 4$

Conflicts During LLG Parsing.

- Syntax Error**
- I) N-Conflict: If $T.O.S$ is non-terminal and table has blank entry, then N-conflict.
- II) T-Conflict: If $T.O.S$ is terminal and $T.O.S \neq$ terminal P/P then T-conflict.

m	t
x	$x \rightarrow \alpha$

If $-T.O.S == x$
 Prediction (Substitution in Lmo)

$\rightarrow$ Pop x
 $\rightarrow$ push α in reverse order

① $S \rightarrow a$ LL(0) CFG ✓
[LL(1), LL(2)...]

② $S \rightarrow aA$ LL(0) CFG ✓
 $A \rightarrow bB$ [LL(1), LL(2)...]
 $B \rightarrow c$

③ $S \rightarrow a|b$ LL(0) ×
LL(1) ✓ and LL(2)...

④ $S \rightarrow a|ab$ LL(0) × LL(1) × LL(2) ✓

⑤ $S \rightarrow a|f|abc|abcde$
LL(4) ✓ (and above)

Note: * Every LL(k) is LL(k+1) CFG
* Every LL(2) CFG is convertible to LL(1) CFG
* Every LL(2) CFG is convertible to LL(1) CFG
* Set of all languages generated by LL(1) CFG

Set of languages generated by LL(2) CFG

Top	Bottom
1. Prediction parser (LL(1) parser)	2. SR parser
2. Predictions (Lma Substitution)	2. Shift and reduced action
3. Use Lma to verify Syntax (to produce Parse tree) or (to derive input)	2. Use "Parse of Pmo"

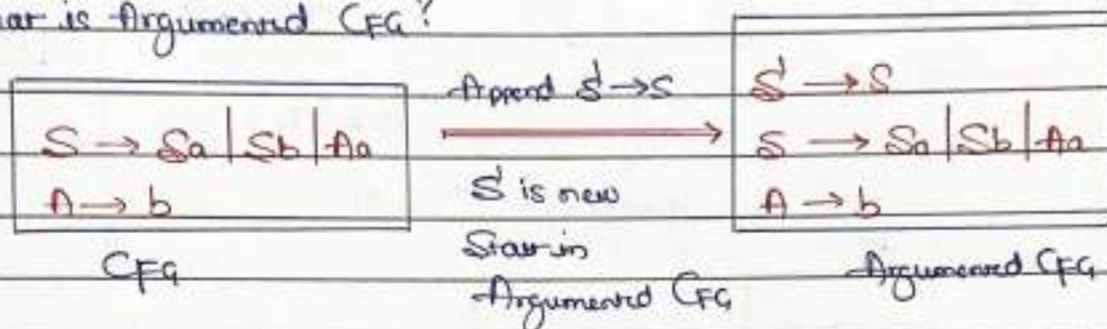
LR(0) Parser:

- no look ahead
Remain reverse
Left to right Scanning
1. LR(0) Parsing diagram [LR(0) DFA]
 2. Conflicts checking for LR(0) CFG
 - SE Conflict
 - RE Conflict
 3. LR(0) Table Construction

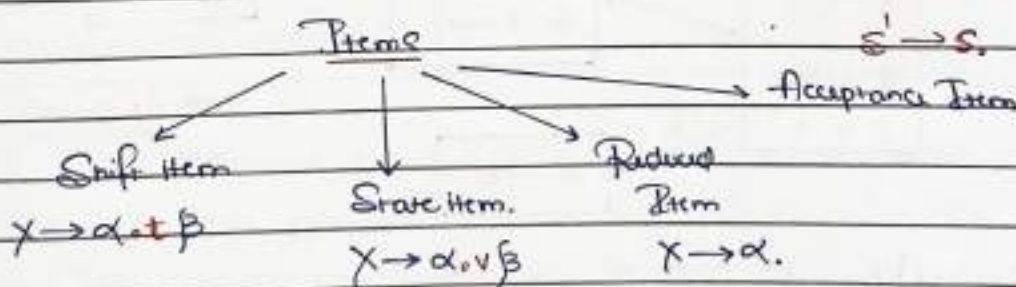
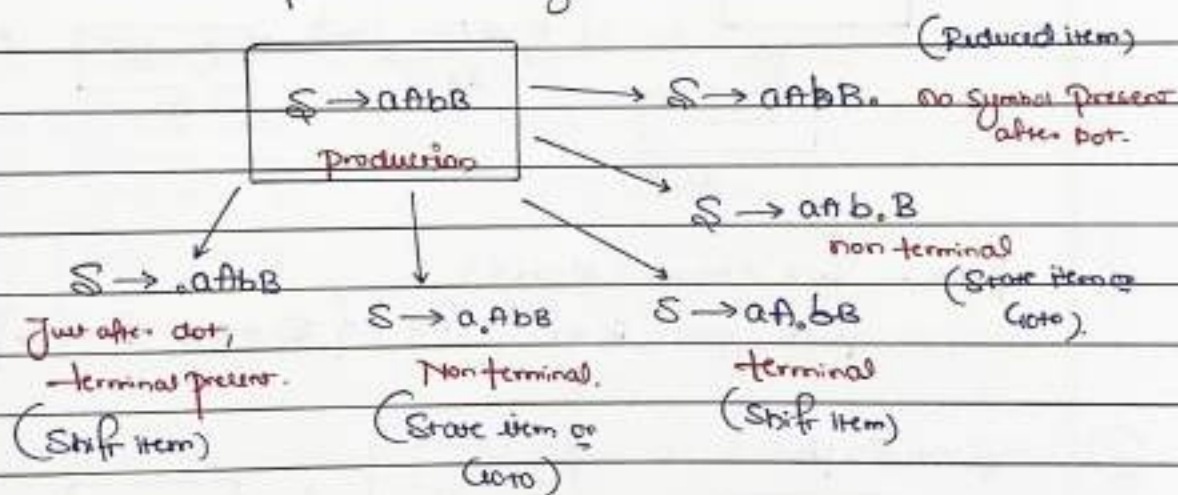
LR(0) DFA:

- 1) Argumented CFG
- 2) Item and types of items?
- 3) Closure () and goto () functions. ***
- 4) How to construct LR(0) DFA?
- 5) How to check SP and PR conflict in LR(0)?

What is Argumented CFG?

LR(0) Item?

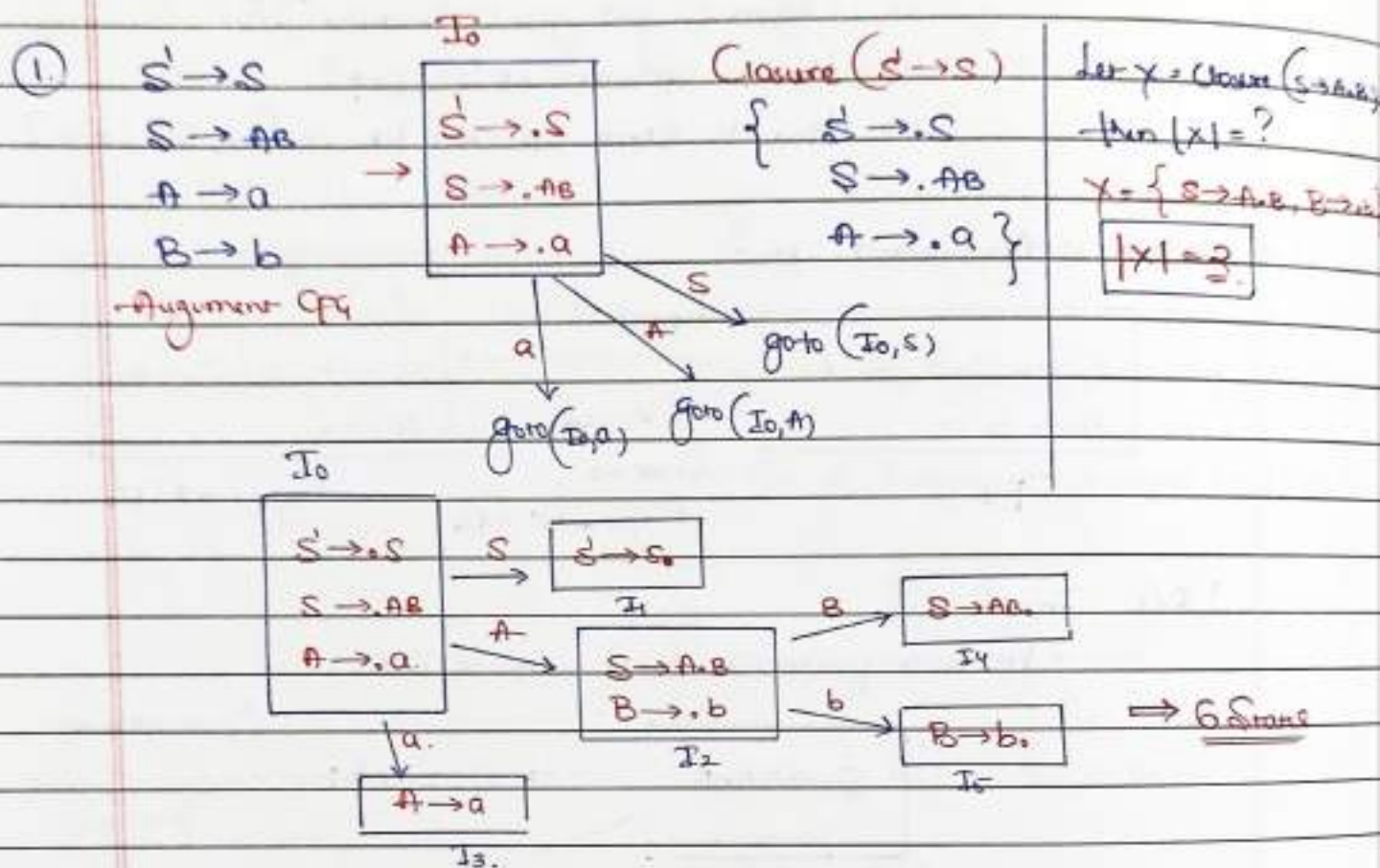
It is a production along with a Dot.

Note:

Dot is used to track progress in derivation (to perform shift and Reduced actions).

Note: * We must focus on "Dot" and next symbol after "Dot".

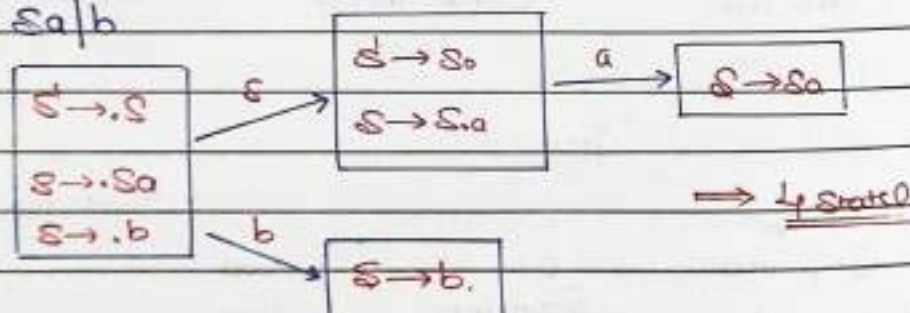
* If non-terminal present just after "Dot" then we have to add all productions in the same state.



Note: $I_0 = \text{Closure}(S' \rightarrow \cdot S)$

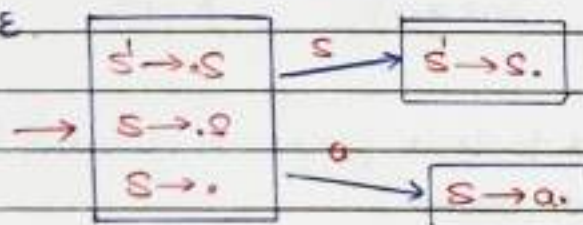
Find $|\text{goto}(I_0, A)| = ? = |\{ S \rightarrow A \cdot B, B \rightarrow \cdot b \}| = 2$.

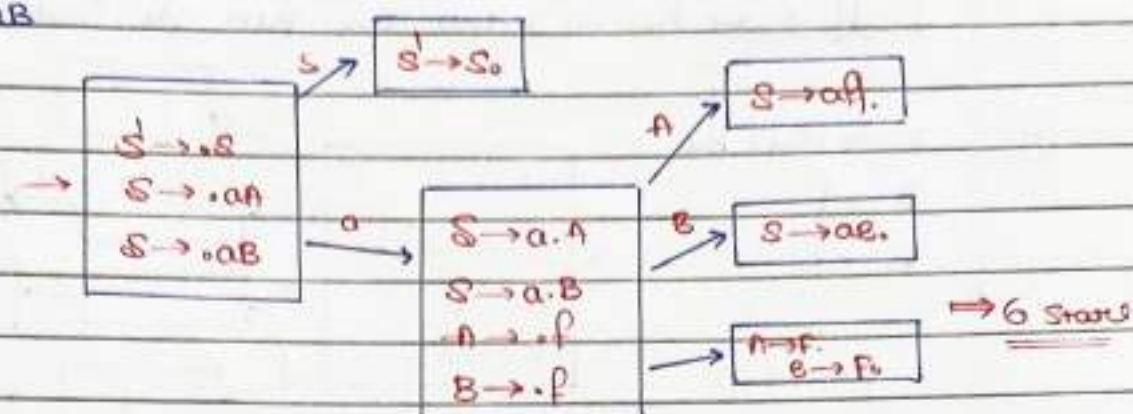
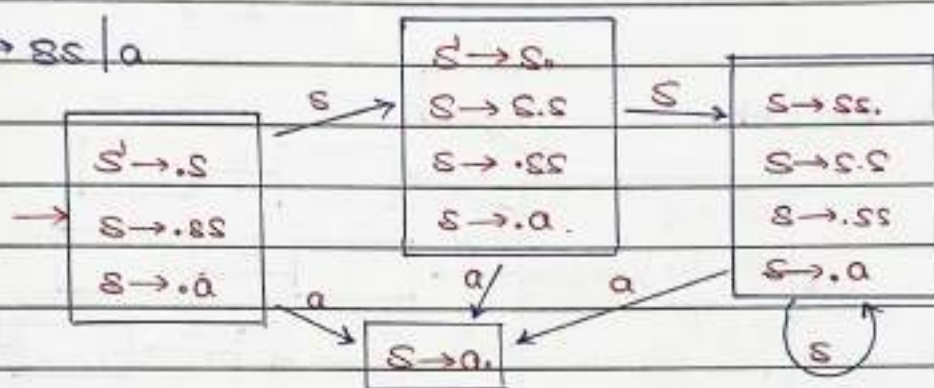
② $S \rightarrow Sa|b$



③

eg. $S \rightarrow a| \epsilon$



(3) $S \rightarrow aA \mid aB$ $A \rightarrow f$ $B \rightarrow f$ (4) $S \rightarrow SS \mid a$ 

How to check given CFG is LR(0) or not?

SR Conflict	RR Conflict
Shift item Reduced item ...	Reduced item Reduced item ...
IF state has both S item and R item then it produces SR conflict	IF State has 2 reduced items then it produces RR conflict

- IF LR(0) DFA has no conflicts then given CFG is LR(0)
- Acceptance item nor participate in any conflict $S' \rightarrow S_0$
- State item also nor participate in any conflict
- State with only one item never produces any conflict
- IF state has reduced item then only there is a possibility to produce either RR or SR conflict.

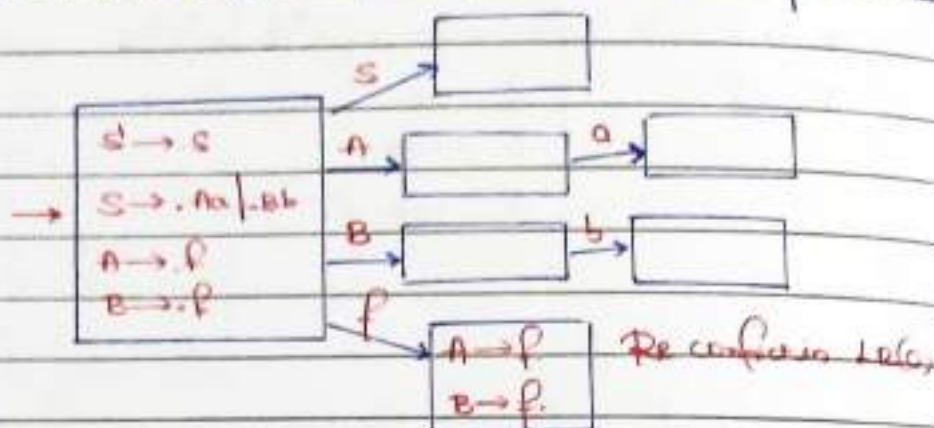
→ If state has no reduce item then the state never produces reduce

⑥ $S \rightarrow Aa \mid Bb$

$A \rightarrow f$

$B \rightarrow f$

Not LR(0)

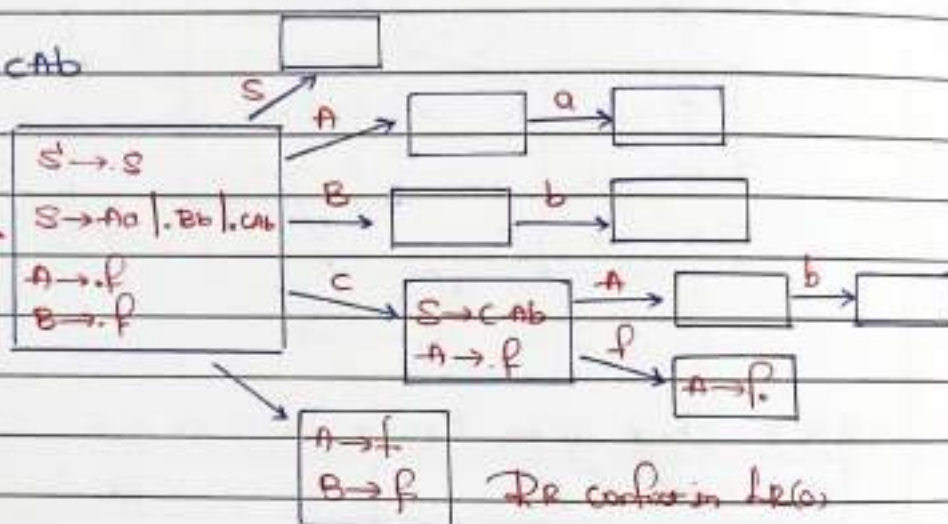


⑦ $S \rightarrow Aa \mid Bb \mid cAb$

$A \rightarrow f$

$B \rightarrow f$

Not LR(0)

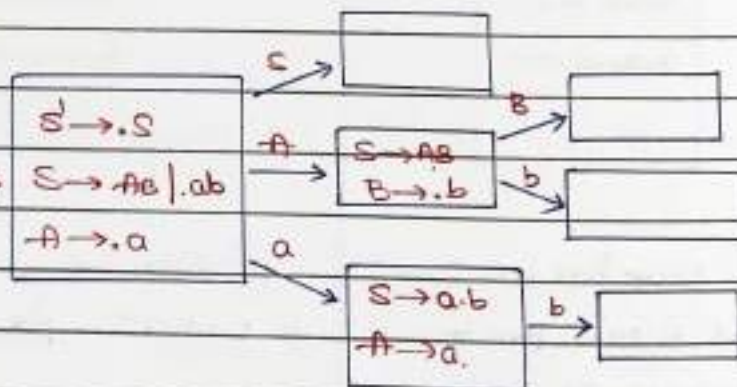


⑧ $S \rightarrow AB \mid ab$

$A \rightarrow a$

$B \rightarrow b$

Not LR(0)



Re conflict in LR(0)

⑨ $S \rightarrow SS \mid (S) \mid \epsilon$

(Not LR(0))

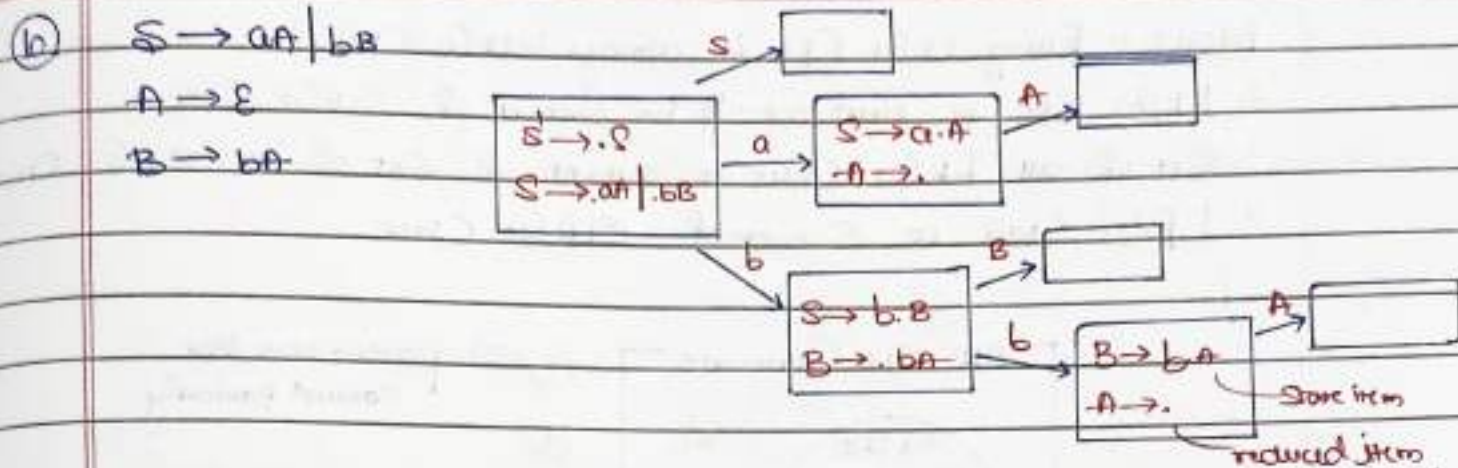
$S' \rightarrow S$
 $S \rightarrow .SS \mid .(S) \mid .$

$S' \rightarrow S$
 $S \rightarrow S.S$
 $S \rightarrow .SS \mid .(S) \mid .$

$S \rightarrow (S)$
 $S \rightarrow S.S$
 $S \rightarrow SS \mid .(S) \mid .$

→ S Indistinguishable states

re conflict



How to check given CFG is SLR(1) or not?

Note: SLR(1) is also called as SLR

Step 1: Construct LR(0) DFA

Step 2: Check conflicts in SLR(1)

Step 3: IF there is no conflict then CFG is SLR(1).

S $\rightarrow$ Simple

L $\rightarrow$ Left to right scan

R $\rightarrow$ reuse of RMD

1 $\rightarrow$ One look ahead

Computed using cube CFG

SR Conflict	RR Conflict
Shift item. Reduced item. ... $X \rightarrow \alpha \cdot \beta$ $Y \rightarrow \alpha \cdot \delta$	Reduced item 1 Reduced item 2 ... $X \rightarrow \alpha$ $Y \rightarrow \alpha$
* IF $\exists \in \text{Follow}(Y)$ then SR conflict in SLR(1)	

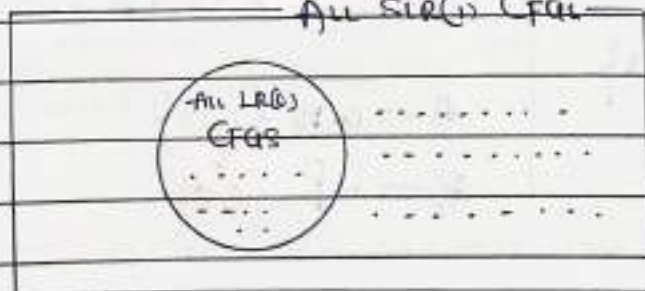
IF $\text{Follow}(X) \cap \text{Follow}(Y) \neq \emptyset$

$\rightarrow$ IF LR(0) DFA has no SLR Conflicts then given CFG is SLR

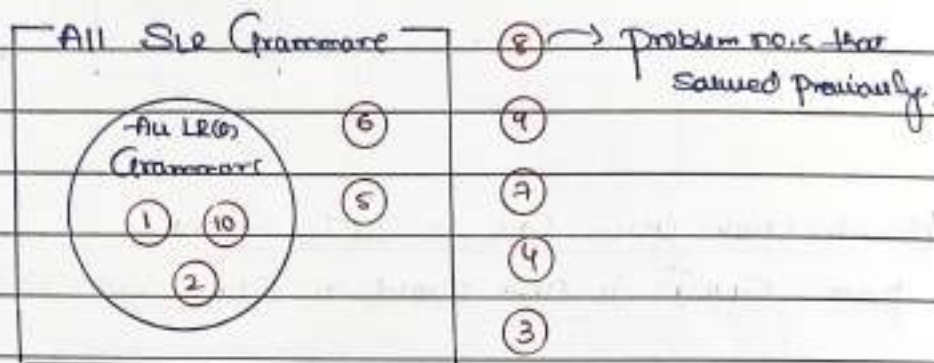
$\rightarrow$ "Terminal of shift item" present in "follow of reduced item".

$\exists \in \text{Follow}(\text{Reduced item})$

All SLR(1) CFGs



- Note:
1. Every LR(0) CFG is always SLR(0) CFG.
 2. LR(0) CFG is need not to be subset of SLR(0) CFG.
 3. Set of all LR(0) CFGs is subset of set of all SLR(0) CFGs.
 4. LR(0) Class is subset of SLR(0) Class.



LR(0) Parser } Depends upon
SLR(0) Parser } LR(0) Items

$X \rightarrow \alpha \cdot \beta$
LR(0) Item

CLR Parser } Depends upon
LR Parser } LR(0) Items

LR(0) Item:
LR(0) Item + Look-ahead set
 $X \rightarrow \alpha \cdot \beta, L$

How to compute Look-ahead Set in LR(0) item?

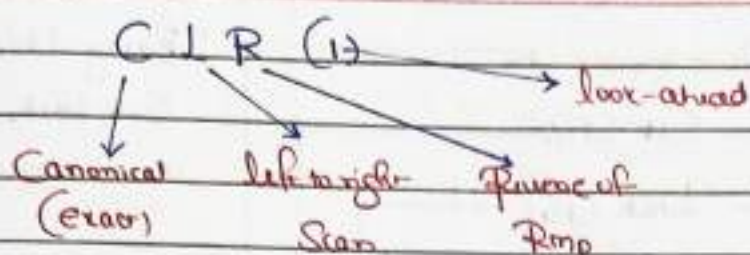
$\square \rightarrow \square \cdot Y \square, L$

$Y \rightarrow \cdot \square$

$X \rightarrow \alpha \cdot Y \beta, L$
 $Y \rightarrow \cdot \gamma, \text{First}(\beta)$

$A \rightarrow a \cdot B b, \{c, d\}$
 $B \rightarrow \cdot e f, \{b\}$

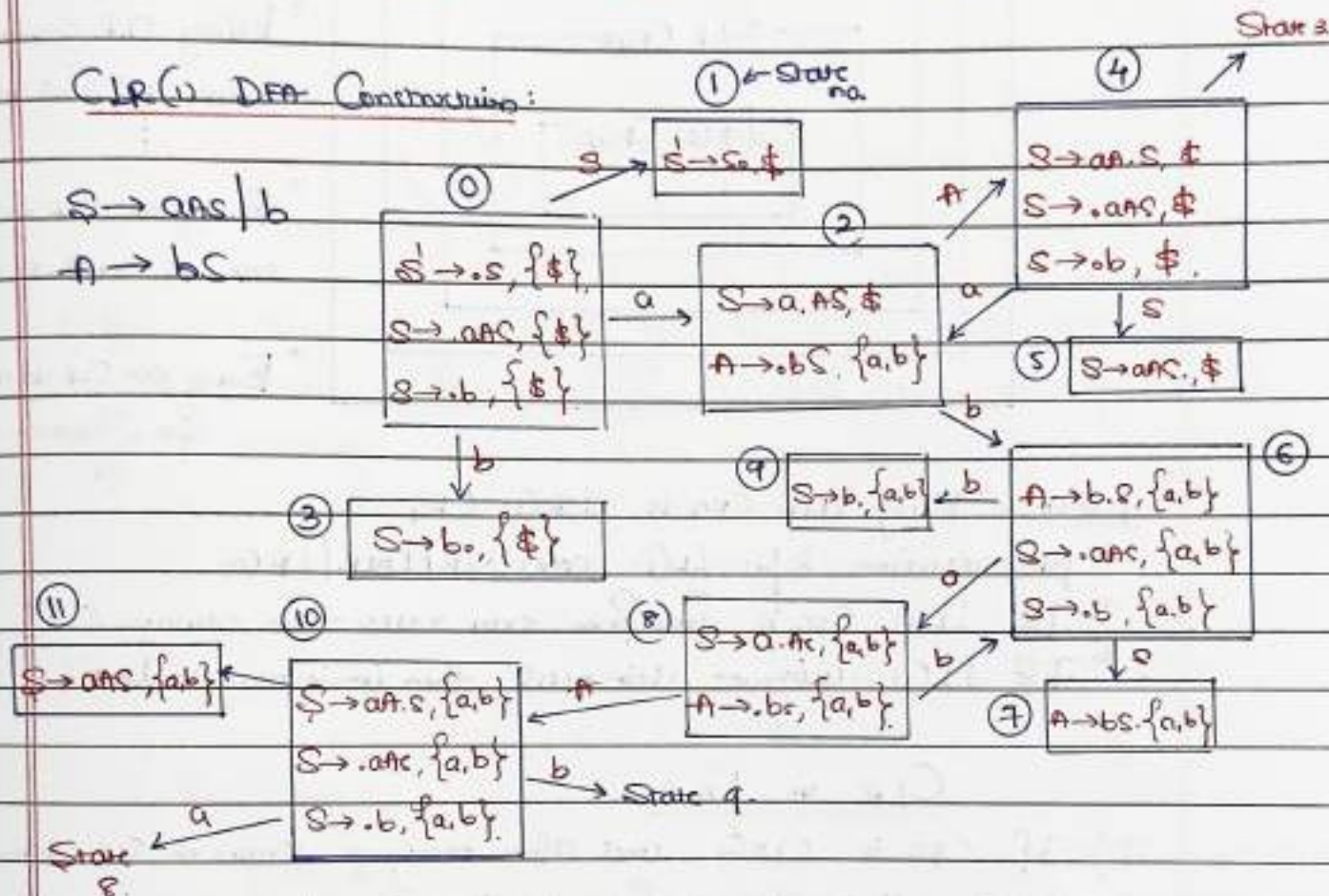
$A \rightarrow a \cdot B, \{c, d\}$
 $B \rightarrow \cdot e f, \{c, d\}$



* CLR(1) is also called as LR(0)

CLR(1) DFA Construction:

1. $S \rightarrow aas | b$
 $A \rightarrow bS$



LA LR(0) DFA : Step 1: Construct CLR DFA

Step 2: Merge state of CLR if they have same item but look-ahead maybe different.

No of State in LR(0)

=
No of State in CLR(1)
=
No of State in LA LR

≤ no. of state in CLR.

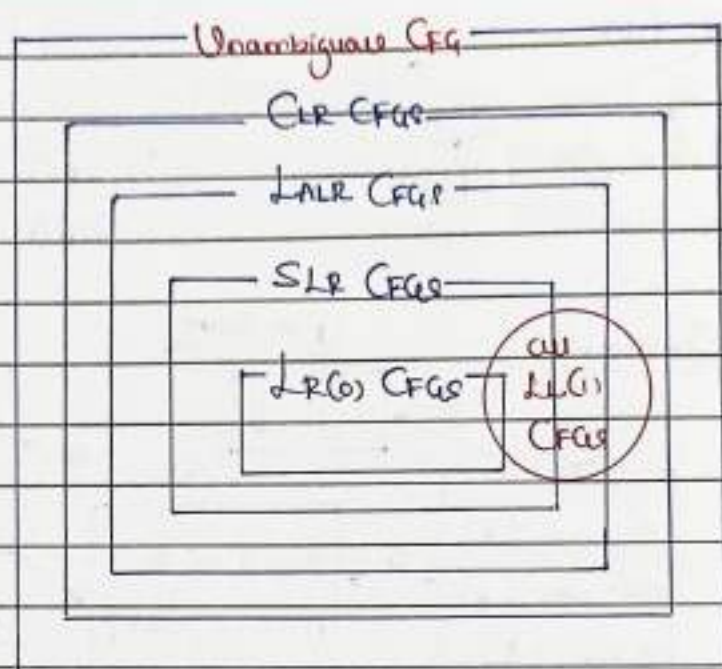
5. $S \rightarrow a | \epsilon \rightarrow S_{LR}, CLR, LA LR$

6. $S \rightarrow aSa | \epsilon \rightarrow \times$

2. $S \rightarrow a \rightarrow LR(0), LR(1), S_{LR}, CLR, LA LR$

3. $S \rightarrow a | ab \rightarrow S_{LR}, CLR, LA LR$

4. $S \rightarrow SS | a \rightarrow \text{Ambiguous}$



* Every LR(0) grammar is SLR, LALR, CLR and Unambiguous.

* Every LR grammar is LALR, CLR, and Unambiguous.

* Every not CLR grammar is not LALR, not SLR, not LR(0).

* Every not LR is not LR(0).
(Same relation for all)

Note: * Every LL(1) CFG is LR(0) CFG

* No relation b/w LL(1) and SLR/LALR/LR(0)

* If LL(1) CFG is free from null rules then always SLR(0)

* If LL(1) without unit rules then it is always LALR(0)

CLR vs LALR

I) If CFG is CLR(0) and after merging states of CLR to make LALR then Re conflict possible in LALR.

$A \rightarrow a, \{t_1\}$
 $B \rightarrow a, \{t_2\}$

S_1 No Re conflict

$A \rightarrow a, \{t_2\}$
 $B \rightarrow a, \{t_1\}$

S_2 No Re conflict

$A \rightarrow a, \{t_1, t_2\}$
 $B \rightarrow a, \{t_2, t_1\}$

S_{12}

Re conflict possible in LALR

II) If CFG is CLR(0) and after merging states of CLR to make LALR then SR conflict never possible in LALR.

$$\begin{array}{l} A \rightarrow a.b, L_1 \\ B \rightarrow a., L_2 \end{array}$$

No SR conflict in LR

 $b \notin L_2$
 $b \notin L_2$ and $b \notin L_4$
 $b \notin L_2 \cup L_4$

$$\begin{array}{l} A \rightarrow a.b, L_3 \\ B \rightarrow a., L_4 \end{array}$$

No SR conflict in LR

 $b \notin L_4$

$$\begin{array}{l} A \rightarrow a.b, L_1 \cup L_3 \\ B \rightarrow a., L_2 \cup L_4 \end{array}$$

No SR conflict in LR

Note: Today, many compilers built based on LR. If any SR conflict occurs, then it performs shift action and proceeds for further parsing.

LR is more popular parser
LR is more powerful parser.

Table Construction:

LL(1) Table:

↓ entry

- I. Production (Rule)
- II. blank

LR Table

↓ entry

- I. Shift entry
- II. Goto entry
- III. Reduced entry
- IV. Acceptance entry
- V. Blank entry.

Shift, reduced, accept
Action Table

Goto Table

Goto entry

Size of Table:

LL(1) Table

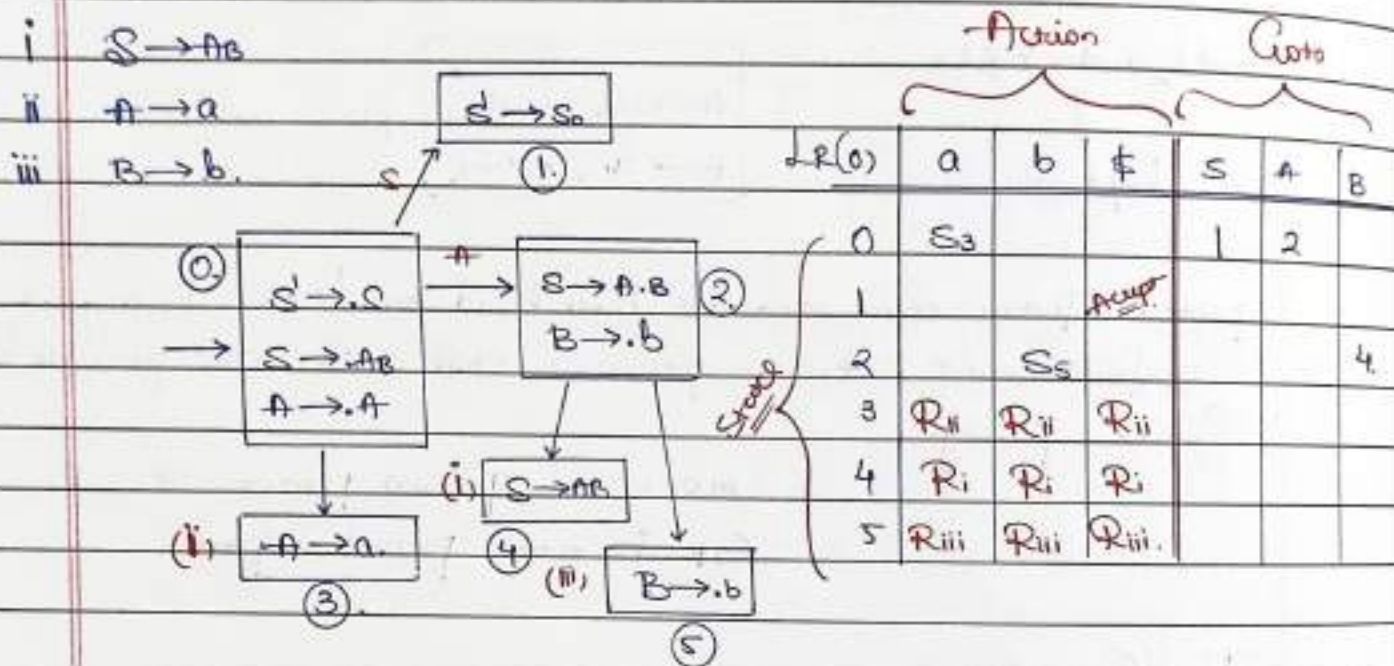
- $|V| * (|T| + 1)$
- No. of non-terminals $\times$ (No. of terminals + 1).

LR Table

- No. of rows $\times$ no. of columns
- Action Table: No. of states $\times$ (no. of terminals + 1)
- Goto Table: No. of states $\times$ (no. of non-terminals)

LR(0) Table Construction:

- Shift entry
- Goto entry
- Reduced entry
- Accept entry



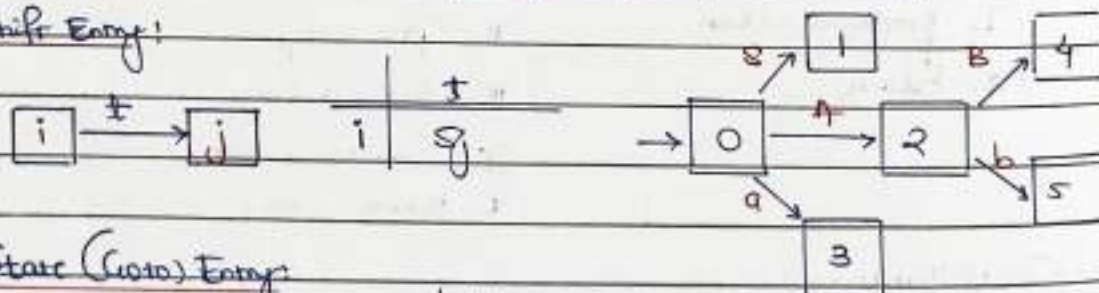
Reduced Entry for LR(0):

→ we should look at every reduced item.

⑦ $A \rightarrow a \cdot$

	Action			Goto		
3	R _{ii}	R _{ii}	R _{ii}			

Shift Entry:



State (Goto) Entry:



Acceptance Entry:



Note:

I. No. of shift entries = No. of terminal transitions in DFA.

II. No. of state entries = No. of non-terminal transitions in DFA.

LR(0) Table & SLR(0) Table

→ Shift entries are same

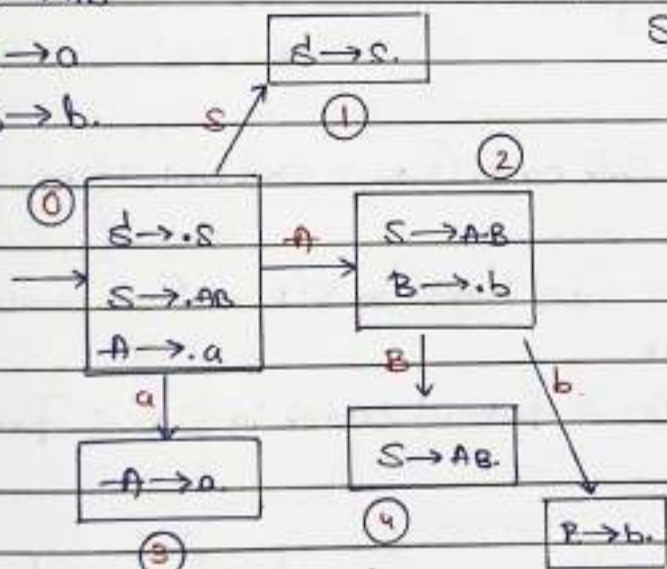
→ Goto entries are same

→ Only reduced entries may differ

Note:

I. No. of shift entries in LR(0) = No. of shift entries in SLR(0)

II. No. of Goto entries in LR(0) = No. of goto entries in SLR(0)

III. No. of reduced entries in LR(0) $\geq$ No. of reduced entries in SLR(0)i. $S \rightarrow AB$ ii. $A \rightarrow a$ iii. $B \rightarrow b$  $Fo(a) = \{b\}$ $Fo(b) = \{\$ \}$ $Fo(\$) = \{\$ \}$ Reduced entry in SLR(0)

$X \rightarrow \alpha \cdot$
i

	t_1	t_2
i	R_1	R_2

 $FOLLOW(X) = \{t_1, t_2\}$

Actions

Goto

SLR(0)	a	b	\$	S	A	B
0	S ₃			1	2	
1			Accept			
2		S ₅				4
3		R ₁				
4			R ₂			
5			R ₃			

CLR and LALR Reduced Entries

→ Look at every reduced item

 $X \rightarrow \alpha \cdot \{t_1, t_2\}$

State i

	t_1	t_2
i	R_1	R_2

(for previous data).

Store

LR(0) Table

- I** Shift entries in LCR , SIR , and LIR are always same

- II. Cost entries in LE (a), SLE, and LATE are always same

- III Reduced series may differ

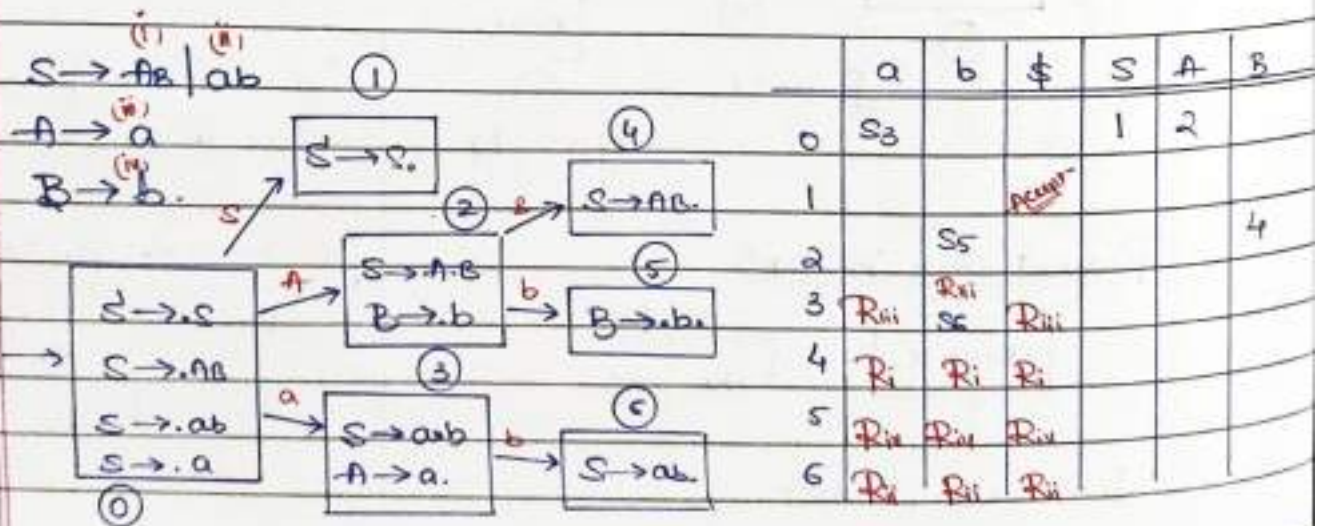
Note:

- + Shift entries $(LR(i)) = \text{Shift entries}(LR) = \text{Shift entries}(LR) \leq \text{Shift entries}$
in $LR(i)$

4. $\text{Cuto entries in LR} = \text{Cuto entries in SLR} = \text{Cuto entries in LALR}$
 $\leq$ Cuto entries in CLR.

- * Reduced entries in LR(0) $\geq$ Reduced entries in SLR $\geq$ Reduced entries in LALR.

eg



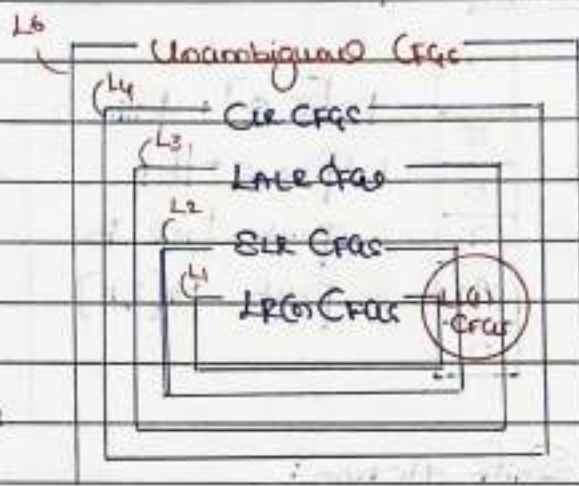
Relation: *more powerful*
I. Parsers: $LR(0) < SLR < LALR < CLR$ *more powerful*
 $LR(1)$
 $LL(1) < LR(0)$

II. Grammar:

$LR(0) \text{ CFG} \xrightarrow{\text{is}} SLR(1) \text{ CFG} \xrightarrow{\text{is}} LALR \text{ CFG} \xrightarrow{F} CLR \text{ CFG} \xrightarrow{\text{is}} \text{Unambiguous}$

III. Classes

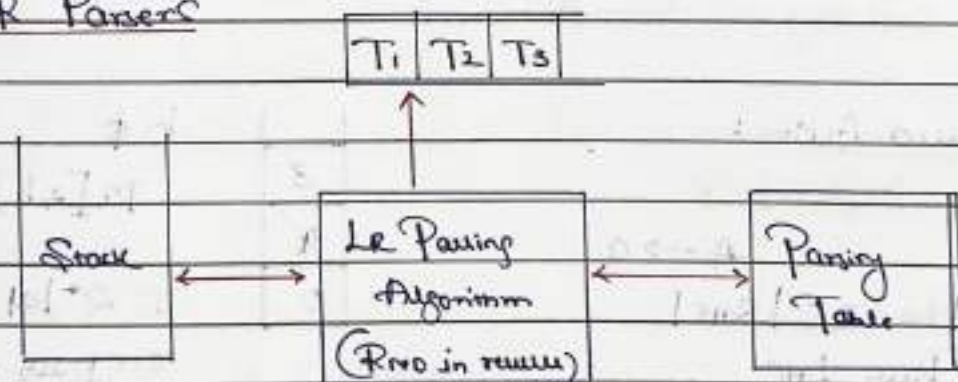
- $L_1 = \text{Set of } LR(0) \text{ CFGs}$
- $L_2 = \text{" SLR "}$
- $L_3 = \text{" LALR "}$
- $L_4 = \text{" CLR "}$
- $L_5 = \text{" } LR(1) \text{"}$
- $L_6 = \text{" Unambiguous CFGs "}$



I $L_1 \subset L_2 \subset L_3 \subset L_4 \subset L_6$

II $L_5 \subset L_4 \subset L_6$

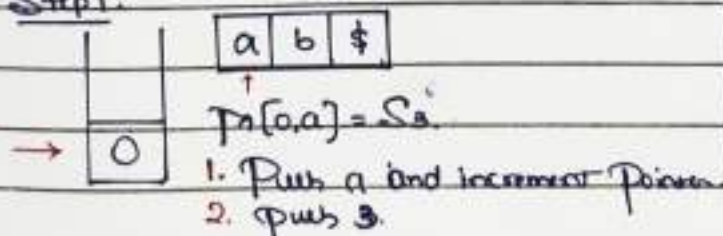
LR Parsers



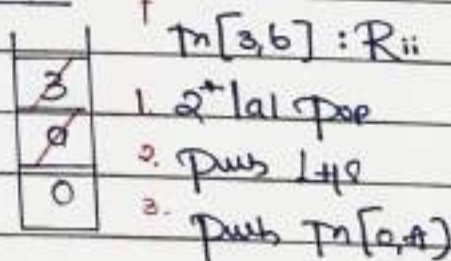
Same for all the parsers.

LR Parsing Algorithm *if p is ab*

Step 1:



Step 2: $b \$$



Steps: $b\$$

2
A
0

 $M[2, b]: S_5$

1. Push b
2. push S

M	a	b	\$	S	A	B
0	S_3			1	2	
1						
2		S_5				4
3			R_{ii}			
4			R_{ii}			
5			R_{iii}			

Step 4: $\$$

5
b
2
A
0

 $M[5, \$]: R_{iii}$

1. Pop $2 + |b|$
2. push LHC B
3. push $M[2, B]$

- (i) $S \rightarrow AB$ (ii) $A \rightarrow a$
(iii) $B \rightarrow b$

Step 6: $\$$

1
5
0

 $M[1, \$]: \text{Accept}$ Shift Action:

	a	3
0	S_3	a
		0

Shift entry

a

↑

- I Push input 'a' onto stack & increment pointer
- II Push next state number

Reduced Action: $M[2, b]: R_{ii}$ $A \rightarrow a$ I Pop $2 + |R_{ii}|$

II Push LHC

Go to entry III. Push to [below, tos.]
tos, (LHC)
(below LHC)

3
A
0

 $b\$$ $M[2, b]: A \rightarrow a$ I. $2 + |a|$

II. Push A

III. $M[0, A]$ push
Go to entry

$$S \rightarrow AB$$

$$W = abb$$

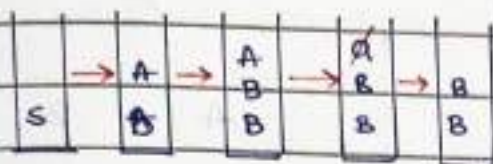
$$A \rightarrow AB/a$$

$$B \rightarrow b$$

Lmo: Smbb

Rmo: Smbb

Reverse of Rmo.



S
↓
AB
↓
ABB
↓
ABBB
↓
abbb
↓
abb

S
↓
AB
↓
Ab
↓
Aeb
↓
Abb
↓
abb

S
↑
AB
↑
Ab
↑
Aeb
↑
Abb
↑
abb

Precedence Relations:

I Higher

II Lowest

III Equal $\rightarrow$ L to R associativity.
R to L associativity.

Higher

$*, /, \%$
Same precedence
$+, -$

 $\rightarrow$ left to right

Lowest

 $\rightarrow$ left to right

① $S \rightarrow A+B$

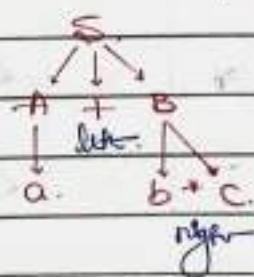
$$A \rightarrow a$$

$$B \rightarrow a+b+c$$

$$B \rightarrow b+c$$

$$a+b+c$$

left right

left
(Stack)

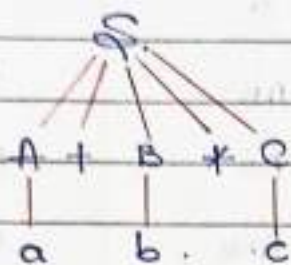
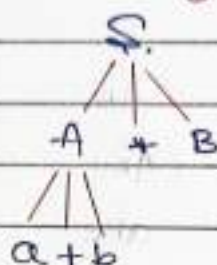
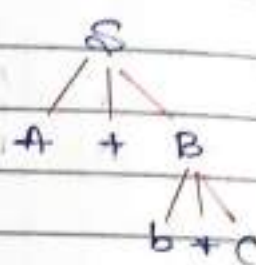
Right (input)

	+	+
	-	<
	*	-

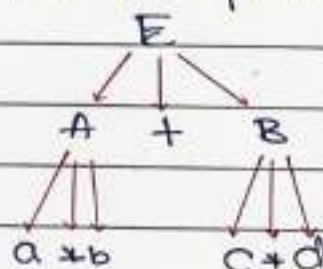
$$a + b * c$$

Left

Right

 $+ = *$  $+ > *$  $+ < *$

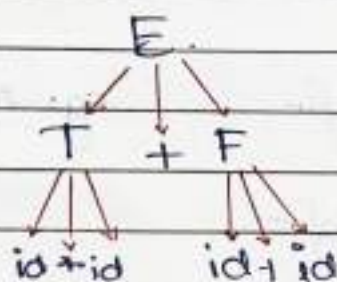
Q1. Find the Precedence using (relating) parse tree



$\rightarrow +$ is lowest
 $\rightarrow *$ is higher.

		Right	
		+	*
Left	+	NR	<
	*	>	NR
		(No relation)	

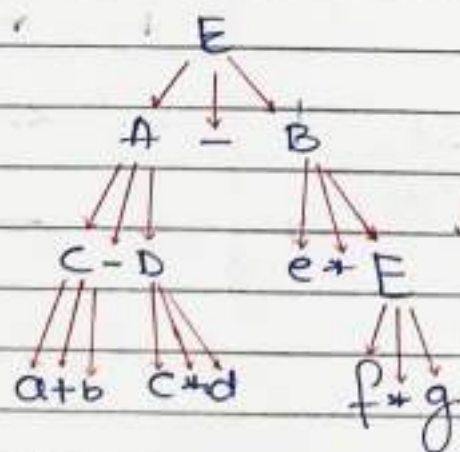
Q2



$\rightarrow +$ is lowest than $*$
 $\rightarrow *$ is higher than $+$
 $\rightarrow +$ is right associative.

		Right	
		+	*
Left	+	<	No Relation
	*	>	No Relation

Q3.



		Right		
		+	*	-
Left	+	X	X	>
	*	X	<	>
	-	X	<	>

→ It may be ambiguous or unambiguous.

- eg: $S \rightarrow A + B$

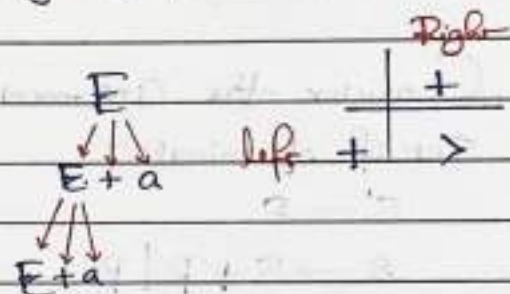
$$\underline{A \rightarrow a}$$

$$B \rightarrow b$$

$$\underline{E \rightarrow E + a \mid a}$$

Life recursion

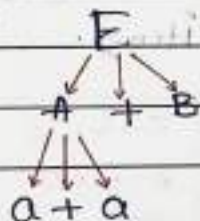
'+' is l.f. associative



Q5. $E \rightarrow A+B$

$$A \rightarrow B + C$$

$$B \rightarrow a$$



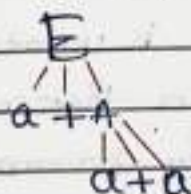
'+' is left associative.

Q6 $E \rightarrow a + E \mid a$

'+' is Right associative.

Q7. $E \rightarrow a + A$

$$A \rightarrow a + a$$



'+' is Right associative.

Q8. $E \rightarrow E + T \mid a$

→ * is higher

$$T \rightarrow F \neq T \mid b$$

→ - is lower

$$\underline{F \rightarrow C}$$

→ + is life associative

→ $*$ is Right associative.

Q9. $E \rightarrow E + E \mid E - E \mid id$

→ + is highest

→ - is lowest

→ + is left-to-right associative

→ - is right-to-left associative

Prqs

Q1. Which of the following statements is true?

Ans. LR(1) parsing is sufficient for deterministic context-free languages.

Q2. Consider the augmented grammar $\{+, *, (,), id\}$ as the set of terminals:

$S' \rightarrow S$

$S \rightarrow S + R \mid P$

$R \rightarrow R * P \mid P$

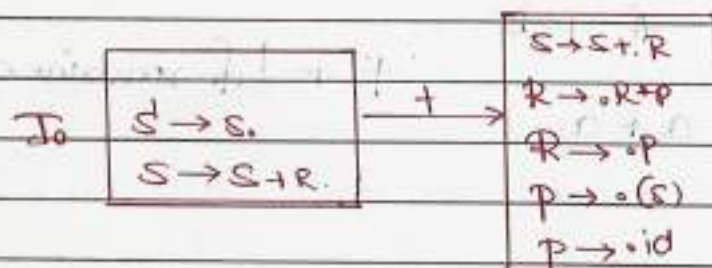
$P \rightarrow (S) \mid id$

If I_0 is set of two LR(1) items

$\{[S' \rightarrow S], [S \rightarrow S + R]\}$, then goto

(closure(I_0), +) contains exactly 5 items.

Ans.



Q3. Consider the following statements:

S_1 : Every LR(1) grammar is unambiguous but there are certain unambiguous grammars that are not LR(1).

S_2 : For any context-free grammar, there is a parser that takes at most $O(n^3)$ time to parse a string of length n .

Which of the following options is correct?

Ans. S_1 and S_2 both are true.

CYK Algorithm

- * Bottom up parsing
- * Dynamic Programming
- * $O(n^3)$ algorithm
- * Membership algorithm for CFG
- * It checks given string generated given CFG or not.

Q4. Consider the following augmented grammar with $\{*, @, <, >, a, b, c\}$ as the set of terminals.

$S' \rightarrow S$	$S \rightarrow <S>$
$S \rightarrow S * c S$	$S \rightarrow \underline{\hspace{1cm}}$
$S \rightarrow S S$	$S \rightarrow \underline{\hspace{1cm}}$
$S \rightarrow S @$	$S \rightarrow \underline{\hspace{1cm}}$
$S \rightarrow <S>$	$S \rightarrow \underline{\hspace{1cm}}$
$S \rightarrow a$	$S \rightarrow * <S>$
$S \rightarrow b$	$S \rightarrow \underline{\hspace{1cm}}$
$S \rightarrow c$	$S \rightarrow \underline{\hspace{1cm}}$

Let $I_0 = \text{CLOSURE}(\{S \rightarrow \cdot S\})$. The number of items in the set $\text{GOTO}(\text{GOTO}(I_0, <), <)$ is 8.

Q5. Consider the following grammar:

$S \rightarrow a S b d$

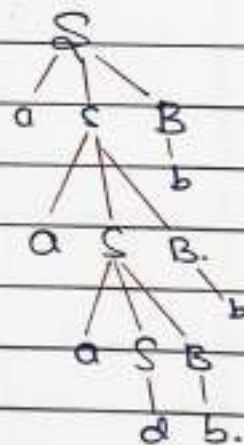
$B \rightarrow b$

The number of reduction steps taken by a bottom-up parser while accepting the string is 7.

No. of reduction steps in BUP (reverse of RMD)

=

No. of steps in RMD



Q6. Which of the following kind of derivation is used by LR parsers?

Ans. Rightmost in reverse.

Q7. Which of the following statements about power is/are correct?

I. Canonical LR is more powerful than SLR.

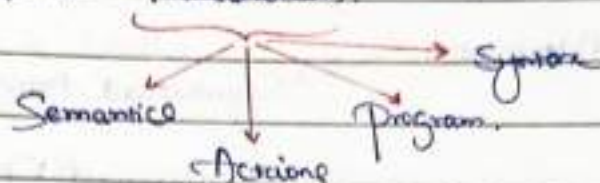
II. SLR is more powerful than LALR.

III. SLR is more powerful than Canonical LR.

Ans. Only I is True.

SYNTAX DIRECTED TRANSLATIONS.

$$\begin{aligned} \text{SDT} &= \text{Syntax} + \text{Translation} \\ &= \text{CFG} + \text{Translation} \end{aligned}$$

SDT Applications:

- It can be used to perform:
 - (i) Semantic analysis
 - (ii) Syntax tree generator
 - (iii) Intermediate code generator
 - (iv) Generating parse trees
 - (v) Any meaningful activity.
- It can be used to translate expressions:

(Infix/prefix/postfix → infix/prefix/postfix)
- It can be used to translate numbers: (Binary/decimal/octal → to any other/decimal/binary)
- It can evaluate expressions.

$$E \rightarrow E + E \{ \text{Translation} \}$$

$$E \rightarrow a. \{ \text{Translation} \}$$

CFG + Translation = SDT.

Lexical	Syntax	Semantic	SDT
Tokens	Structures: → declarative Syntax → if, if else Syntax → loop Syntax → function Syntax → Expression Syntax	Type checking → in expression → in functions → declare before use of variable	Anything that has some logical information in syntax. More powerful than Compiler.

Syntax Directed Translations:

① Attribute $\rightarrow$ Inherited Attribute
 $\rightarrow$ Synthesized Attribute

② Definitions $\rightarrow$ L. Attributed EDT
 $\rightarrow$ S. attributed SDT

③ Evaluation

eg: $S \rightarrow a$ $\{ S.x = a.val \}$
 $\downarrow$ $\downarrow$ $\downarrow$
 CFG Translation Attribute (x, val)

eg: $S \rightarrow AaB$ S : Parent of A, a, B
 A : Child of S, left sibling of a and B
 a : Child of S, right sibling of A, left sibling of B
 B : Child of S, right sibling of A and a

Production tree

Q1. $E \rightarrow T + F \{ E.x = T.y * F.z \}$

Where is the computation?

Ans "E", it is computed depending on its children, T and F.



Attributes:

① Inherited Attribute: Computation depends on parent/siblings

eg: $S \rightarrow AB \{ A.x = B.y \}$

x is inherited attribute

② Synthesized Attribute: Computation depends upon children

eg: $E \rightarrow a \{ E.x = a.val \}$

Identify the type of attribute:

① (i) $E \rightarrow E_1 + E_2 \{ E.x = E_1.x + E_2.x \}$

(ii) $E \rightarrow id \{ E.x = id.val \}$

Depends On Child

→ In Rule (i), x is Synthesized attribute

→ In Rule (ii), x is Synthesized attribute

→ In SOT, x is Synthesized attribute

② I. $S \rightarrow D L; \{ L.type = D.type \}$

II. $D \rightarrow int \{ D.type = int \}$

III. $L \rightarrow L_1 id \{ L_1.type = L.type \}$

IV. $L \rightarrow id \{ \}$

→ In Rule I, type is inherited attribute

→ In Rule II, type is Synthesized attribute

→ In Rule III, type is inherited attribute

→ In whole SOT, type is neither inherited nor synthesized

③ $S \rightarrow S_1 S_2 \{ S.count = S_1.count + S_2.count \}$

$S \rightarrow (S_1) \{ S.count = S_1.count + 1 \}$

$S \rightarrow \epsilon \{ \}$

→ Count is Synthesized attribute in above SOT

④ $S \rightarrow Aa \{ S.x = Ax; A.y = Sx \}$

$A \rightarrow b \{ A.x = 10; A.y = 100 \}$

→ x is Synthesized attribute

→ y is inherited attribute

⑤ $E \rightarrow T + F \{ E.x = T.y; F.y = E.x + 2; E.y = F.y - 1 \}$

$T \rightarrow id \{ T.x = 10; T.y = id.val \}$

$F \rightarrow id \{ F.x = id.val; F.y = 20 \}$

→ x is Synthesized attribute

→ y is neither inherited nor synthesized attribute

L-Attributed SOT	S-Attributed SOT
* Computation depends on parent/left sibling/children. * Translation can be placed anywhere in production. * Evaluation depends upon translation order from left to right.	* Computation depends on only children. * Translation should be at the end of production. * Evaluation depends on bottom up parsing. (Reverse of RMO) (Non-terminal Order)

Note: * Every S-attributed grammar is L-attributed.

* L-attributed SOT may or may not be S-attributed.

Identify the type of attribute:

① (i) $E \rightarrow E_1 + E_2 \{ E.x = E_1.x + E_2.x \}$

(ii) $E \rightarrow id \{ E.x = id.val \}$

→ The above SOT is both L and S-Attributed.

② $S \rightarrow D, L \{ L.type = D.type \}$

$D \rightarrow int \{ D.type = int \}$

$L \rightarrow L_1, id \{ L.type = L_1.type \}$

$L \rightarrow /id \{ \}$

depends on parent.

→ The given SOT is L-attributed
but not S-attributed.

③ $S \rightarrow S_1 S_2 \{ S.count = S_1.count + S_2.count \}$

$S \rightarrow (E) \{ S.count = S_1.count + 1 \}$

$S \rightarrow \epsilon \{ \}$

Above SOT is both 'L' and 'S' attributed grammar

MCQ.

- A. L attributed
B. S attributed
C. L attributed but not S
D. NONE.

$$④ \quad S \rightarrow Aa \{ S.x = A.x; A.y = S.x \}$$

$$S \rightarrow b \{ A.x = 100; A.y = 1000 \}$$

→ SOT is not S-attributed

→ SOT is L-attributed

$$⑤ \quad E \rightarrow T+F \{ E.x = T.y; T.y = E.x+2; E.y = F.y-1 \}$$

$$T \rightarrow id \{ T.x = 10; T.y = id.val \}$$

$$F \rightarrow id \{ F.x = id.val; F.y = 20 \}$$

→ Above SOT is L-attributed but not S-attributed

$$⑥ \quad S \rightarrow S_1 S_2 \{ S_1.x = S_2.x + S_1.x \}$$

$$S \rightarrow a \{ S.x = a.val \}$$

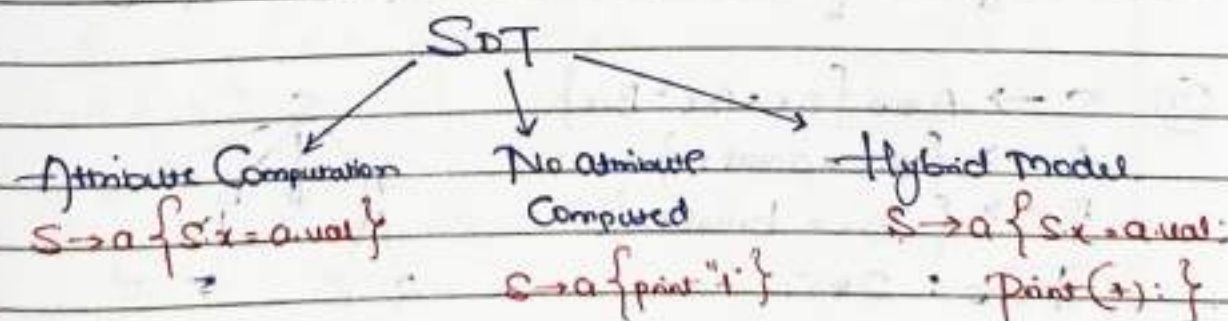
→ Above SOT is neither S-attributed nor L-attributed

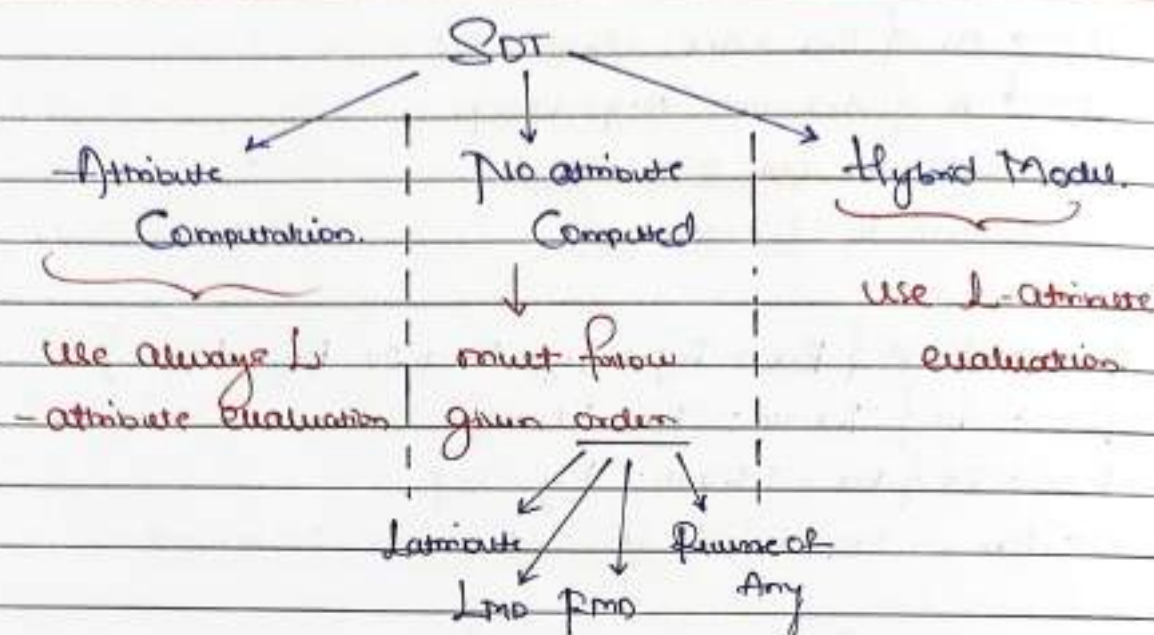
$$⑦ \quad S \rightarrow \{ S.x = a.val \}$$

$$S \rightarrow b \{ S.x = b.val \}$$

→ Above SOT is L-attributed but not S-attributed

L-attributed SOT	S-attributed SOT
<u>Evaluation (Translation)</u> : All translational evaluation from left to right. * (Topological order) * (In order) * (DFS, left to right)	<u>Evaluation (Translation)</u> : Evaluation on non-terminals depends on reverse of Pmo. * (Bottom up parsing) * (LR parsing) * (SR parsing)





① $S \rightarrow A \# B \{ S.x = A.x + B.x \}$

$A \rightarrow a \{ A.x = a.val \}$

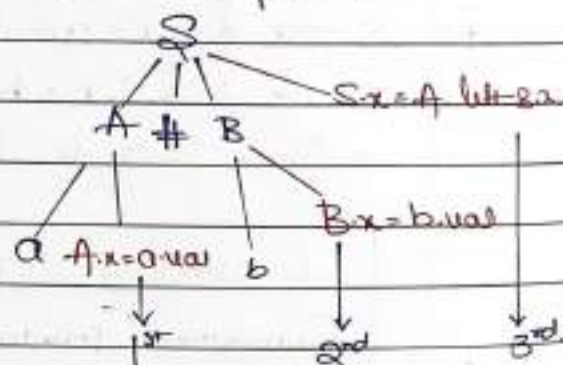
$B \rightarrow b \{ B.x = b.val \}$

Input: $10 \# 3$

Method I: using L-attribute evaluation.

$\Rightarrow 13$

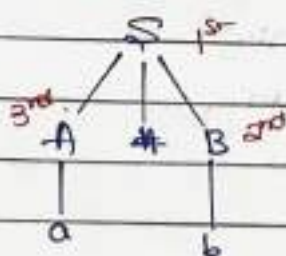
(Decorated PT)
Annotated parse tree



Method II: using R-attribute evaluation
(Reverse of Rmo)

$\Rightarrow 13$

Rmo.



Rmo numbering: $S \ B \ A$

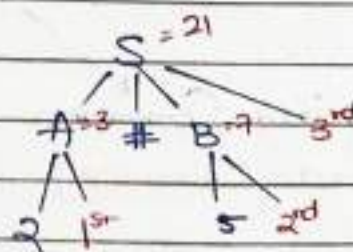
Reverse of Rmo: $A \ B \ S$

② $S \rightarrow A \# B \{ S.x = A.x * B.x \}$

$A \rightarrow \{ A.x = a.val + 1 \}$

$B \rightarrow \{ B.x = b.val + 2 \}$

Input: $2 \# 5$



$$(3) S \rightarrow S S_2 \{ S.x = S_1.x - S_2.x \}$$

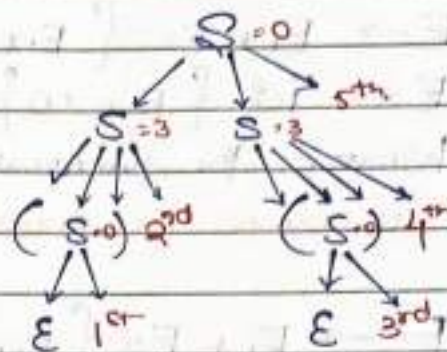
$$S \rightarrow (S_1) \{ S.x = 3 + S_1.x \}$$

$$S \rightarrow \epsilon \{ S.x = 0 \}$$

Input = () ()

Multiple Answers

$\therefore$ Ambiguity



and many

Other

different

Answers

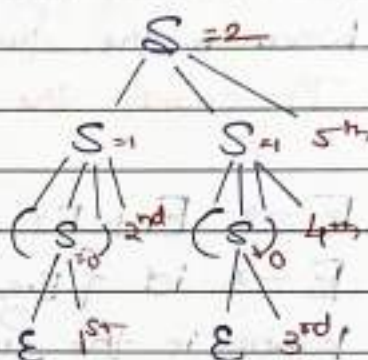
$$(4) S \rightarrow S S_2 \{ S.x = S_1.x + S_2.x \}$$

$$S \rightarrow (S_1) \{ S.x = 1 + S_1.x \}$$

$$S \rightarrow \epsilon \{ S.x = 0 \}$$

Input: () () + () () + () ()

$\Rightarrow 2$



$$(5) S \rightarrow S S_2 \{ S.x = S_1.x + S_2.x \}$$

$$S \rightarrow (S_1) \{ S.x = 1 + S_1.x \}$$

$$S \rightarrow \epsilon \{ S.x = 0 \}$$

What is the functionality of given SPT?

Ans It counts no. of balanced parentheses at root.

$$(6) S \rightarrow S S_2 \{ S.x = S_1.x + S_2.x \}$$

$$S \rightarrow (S_1) \{ S.x = 2 + S_1.x \}$$

$$S \rightarrow \epsilon \{ S.x = 0 \}$$

What is the functionality of the given SPT?

Ans It counts number of (parentheses) terminals in the input (length of input)

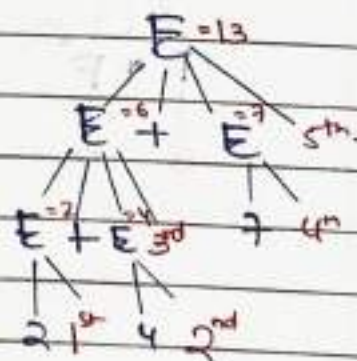
$$(7) E \rightarrow E_1 + E_2 \{ E.x = E_1.x + E_2.x \}$$

$$E \rightarrow id \{ E.x = id.val \}$$

Input: 2 + 4 + 7

What is attribute value computed at root?

Ans 13



- ⑧ $E \rightarrow E_1 + E_2 \{ E.x = E_1.x + E_2.x + 1 \}$
 $E \rightarrow id \{ E.x = 0 \}$

What is the functionality of the given SOT?

Ans It counts the number of Operators.

- ⑨ $E \rightarrow E_1 + E_2 \{ E.x = E_1.x + E_2.x \}$
 $E \rightarrow id \{ E.x = 1 \}$

What is the functionality of the given SOT?

Ans It counts the number of Operands.

- ⑩ $E \rightarrow \sim E_1 + E_2 \{ E.x = E_1.x + E_2.x + 1 \}$
 $E \rightarrow id \{ E.x = 1 \}$

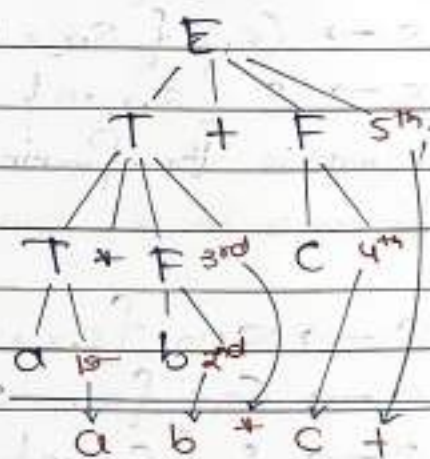
What is the functionality?

Ans Length of input (no. of terminals, Operands & Operators).

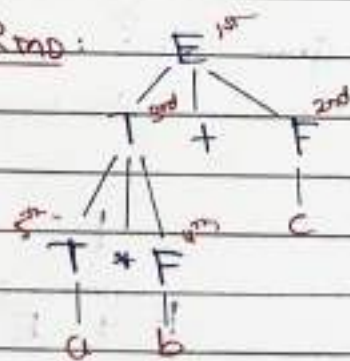
- ⑪ $E \rightarrow T + F \{ \text{print } + \}$
 $E \rightarrow id \{ \text{print id.val} \}$
 $T \rightarrow T * F \{ \text{print } * \}$
 $T \rightarrow id \{ \text{print id.val} \}$
 $F \rightarrow id \{ \text{print id.val} \}$

Input: $a * b + c$

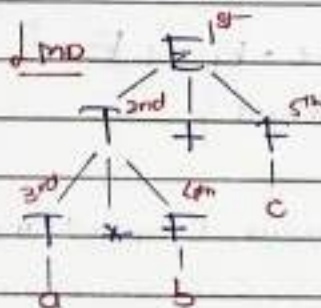
Output: $ab * c +$



Reverse:

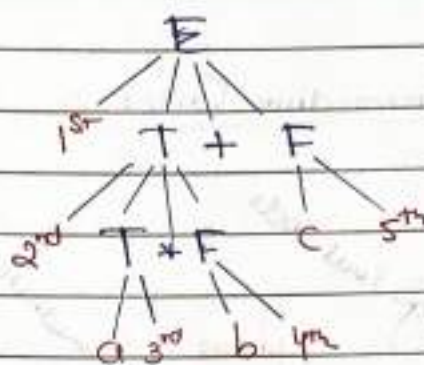


Reverse of Rmo: $ab * c +$



Reverse of Rmo: $ab * c +$

- (12) $E \rightarrow \{ \text{print } + \} T + F$
 $E \rightarrow \text{id} \{ \text{print id.val} \}$
 $T \rightarrow \{ \text{print } * \} T * F$
 $T \rightarrow \text{id} \{ \text{print id.val} \}$
 $F \rightarrow \text{id} \{ \text{print id.val} \}$
 Input: $a + b + c$



Translation Order $\longrightarrow$
 1st 2nd 3rd 4th 5th
 $+ * a b c$

(i) What is the output?

Ans $+ * abc$

(ii) What is output using bottom up parsing?

Same as Q11. bottom up parsing

Note:

1. Attribute evaluation depends on translation order! (If translation position changes then o/p may change.)
2. Top and Bottom evaluations depend on non-terminal order. (If translation position changes then o/p will change remain same as previous.)

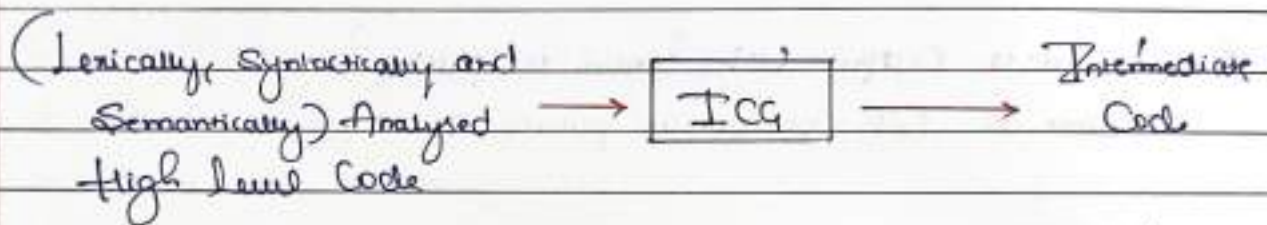
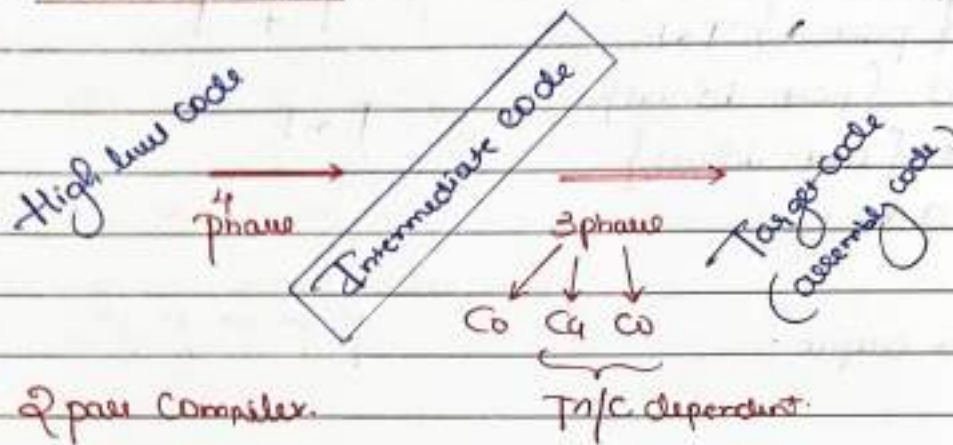
$$a + b + c = 12$$

$$a + b + c = 12$$

$$a + b + c = 12$$

INTERMEDIATE CODE & CODE OPTIMIZATION

Intermediate Code:



Intermediate Code Representations (ICR):

- ① Linear Form $\rightarrow$
 - (i) postfix form
 - (ii) 3AC/TAC/Three address code
 - (iii) SSA Code (Static Single Assignment)
- ② Non Linear Form $\rightarrow$
 - (i) Syntax tree
 - (ii) Directed Acyclic graph
 - (iii) Control flow graph

eg: $x = a + b * c$

Postfix code: $x = a + b * c$

$\Rightarrow x \quad a \quad b * c \quad +$

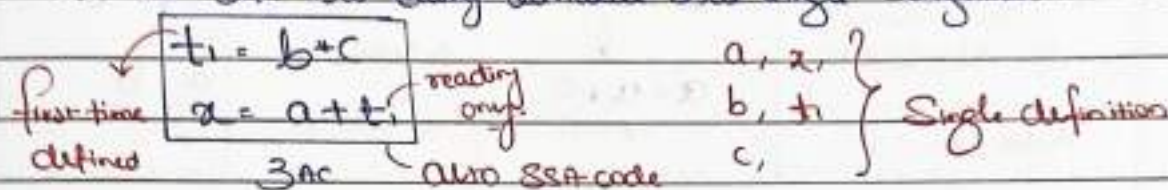
Three-Address Code:

$\rightarrow$ Every Statement containing atmost 3 address/variables (variables)

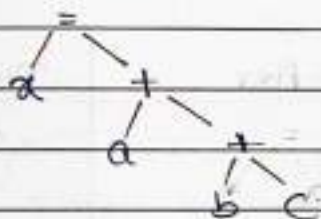
$t_1 = b * c$	$b = b * c$	$c = b * c$
$x = a + t_1$	$b = a + b$	$a = a + c$
3AC	3AC	3AC
5 variables	3 variables	3 variables

SSA Code: (Static Single-Assignment)

→ It is 3AC but every variable has single assignment.

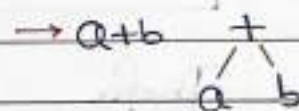


Syntax tree: (Abstract Syntax Tree)



leaf: Operand

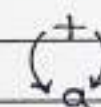
Non leaf: Operator



Directed Acyclic Graph (DAG)

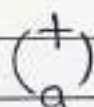
→ It eliminates common sub expressions.

$a + b$



DAG

or



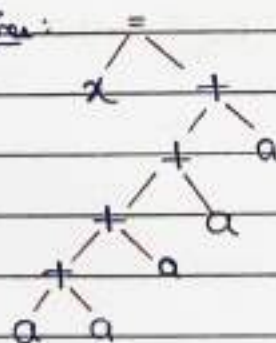
DAG

2 nodes

2 edges

eg: $x = a + a + a + a$

Syntax Tree:



DAG



3 nodes

4 edges

① $x = a + b - b + c$

Postfix	3AC	SSA code	Syntax Tree	DAG
$x = ((a+b)-b)+c$ $\underbrace{a+b}_{ab+}$ $\underbrace{ab+-}_{ab+b-}$ $\underbrace{ab+b-c}_{ab+b-c-}$ $xab+b-c+=$	With min no. of variables $x = a + b - b + c$ $x = a + c$ $\downarrow$ $a = a + c$ 2 variables	min. no. of variables $a_1 = a + c$ 3 variables		min nodes min edges

② $x = (a+b) * (a+b)$

Postfix	3AC	SSA	AST	DAG
$xab+ab*+=$	$a = a + b$ $a = a * a$ 2 variables	$a_1 = a + b$ $a_2 = a_1 * a_1$ 4 variables		4 nodes 4 edges

③ $x = a * b + a + b * c;$

3AC
Find min no of variables
 $x = (a * b) + a + (b * c)$
 $= (b * a) + (b * c) + a$
 $= b * (a + c) + a$

$$\begin{aligned} c &= a + b \\ b &= b * c \\ b &= b + a \end{aligned}$$

3 variables

SSA
Find min no of variables

$$\left. \begin{aligned} c_1 &= a + c \\ b_1 &= b * c_1 \\ b_2 &= b_1 + a \end{aligned} \right\} 6 \text{ variables}$$

(4)

$$x = a + b * c - b$$

$$C = b * c$$

$$C = a + C$$

$$C = C - b$$

3 AC

3 variables

$$C_1 = b * c$$

$$C_2 = a + C_1$$

$$C_3 = C_2 - b$$

5 AC

6 variables

Three Address Code Notations:

- * Triple notation
- * Quadruple notation
- * Indirect triple notation:

Advantage: Less Space

Disadvantage: Computation

takes much time

eg:

$$x = a + b$$

$$y = x * c$$

$$z = x + y$$

$$w = z$$

Triple Notation

	operator	operand 1	operand 2
1000	+	a	b
1010	*	1000	c
1020	+	1000	1010
1030	=	1020	

1000 : (+, a, b)

1010 : (*, 1000, c)

1020 : (+, 1000, 1010)

1030 : (=, 1020)

Quadruple Notation:

	Operator	Operand 1	Operand 2	result
1000	+	a	b	x
1015	*	x	c	y
1030	+	x	y	z
1045	=	z		w

Advantage: Takes less time to compute

Disadvantage: More Space

eg:

	Indirect Triple Notation				Indirect	Actual Address
		operator	operand1	operand2		
$a = a + b$					6000	1000
$y = x * c$					7000	1010
$z = x + y$	1000	+	a	b	8000	1020
$w = z$	1010	*	6000	c	9000	1030
	1020	+	6000	7000		
	1030	=	8000			

Three Address Code for 'if/else' Statement:

<pre> if (x < y): z = x else: z = y; z = z * 2; </pre>	<pre> - to = x < y; Ifz - to Goto - l0; z = x; Goto - l1; - l0: z = y; - l1: z = z * 2; </pre>
---	---

Three Address Code for 'while' Statement:

<pre> while (x < y) { x = x * 2; } y = x; </pre>	<pre> - l0: - to = x < y; Ifz - to Goto - l1; x = x * 2; Goto - l0; - l1: y = x; </pre>
---	--

Three Address Code for Array:

Assume declarations: $A[n_1, n_2]$

3AC for $A[i, j]$ is:

$$-l_1 = n_2 * i$$

$$t_2 = t_1 \times j \quad \rightarrow A[i, j] = \text{Base address} + (n_2 \times i \times j) \times w$$

$$t_3 = t_2 \times w \quad \text{Here, } n_2 \text{ is size of each row and}$$

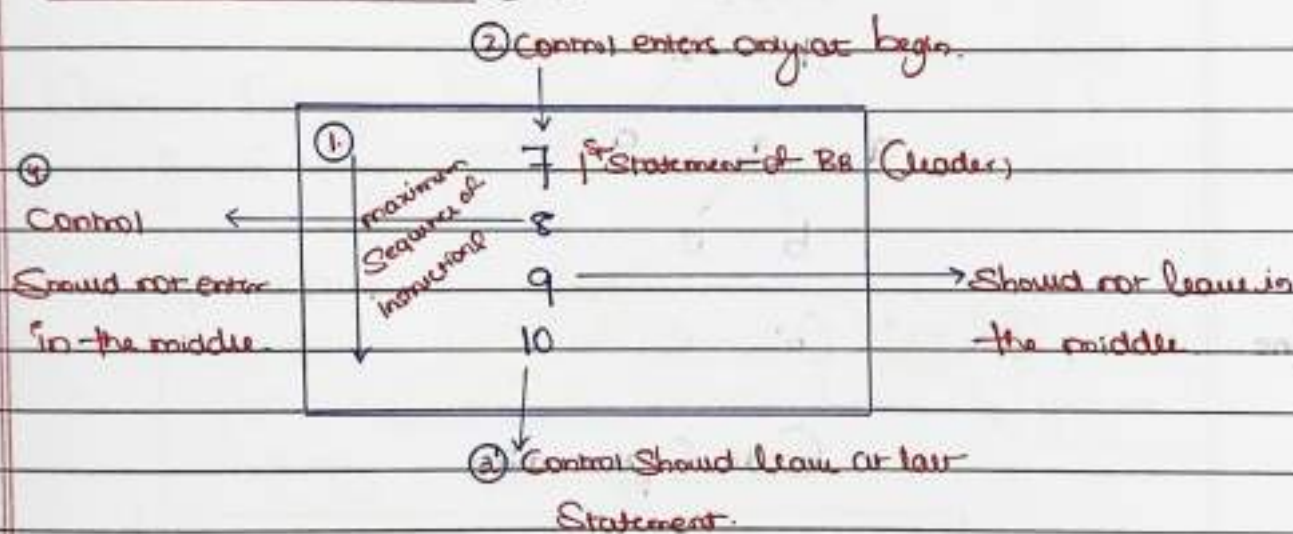
$$t_4 = \text{Base address} \quad w \text{ is size of each element.}$$

$$t_5 = t_4[t_3]$$

Control Flow Graph:

- * It comprises of nodes and edges.
- * It is collection of basic blocks and control.
- * It represents flow of program execution using basic blocks.

What is basic block (BB)?



```

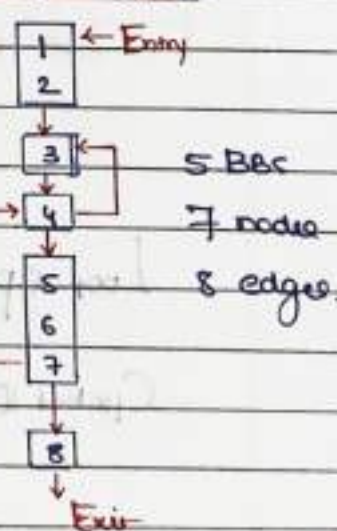
1. x = a + b
2. y = x + y
3. z = x + y
4. if (z > d) goto 3
5. z = x - y
6. z = z + a
7. if (z > 1000) goto 4
8. print z

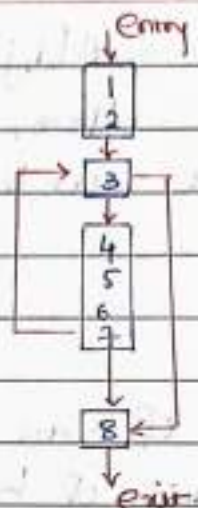
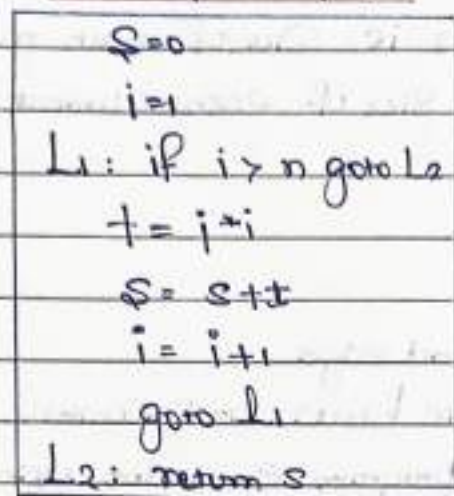
```

Flow



Control Flow Graph



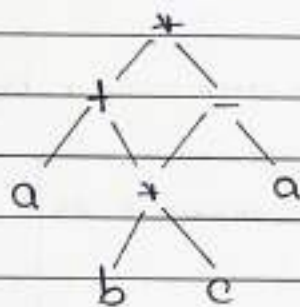
Control Flow Graph

No of B.B.s = 4

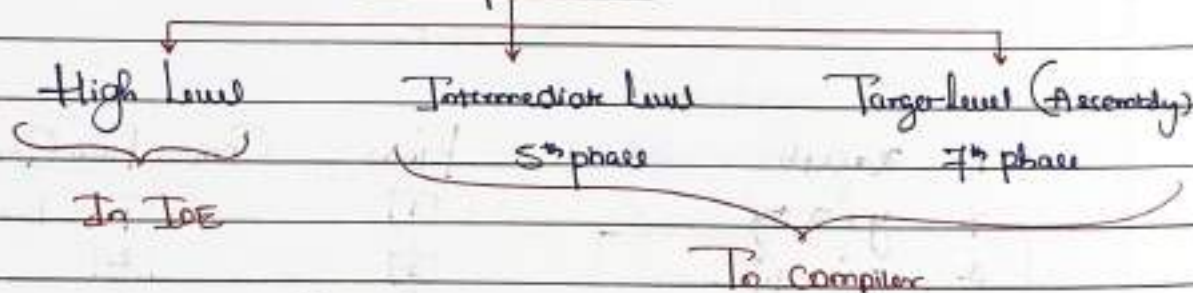
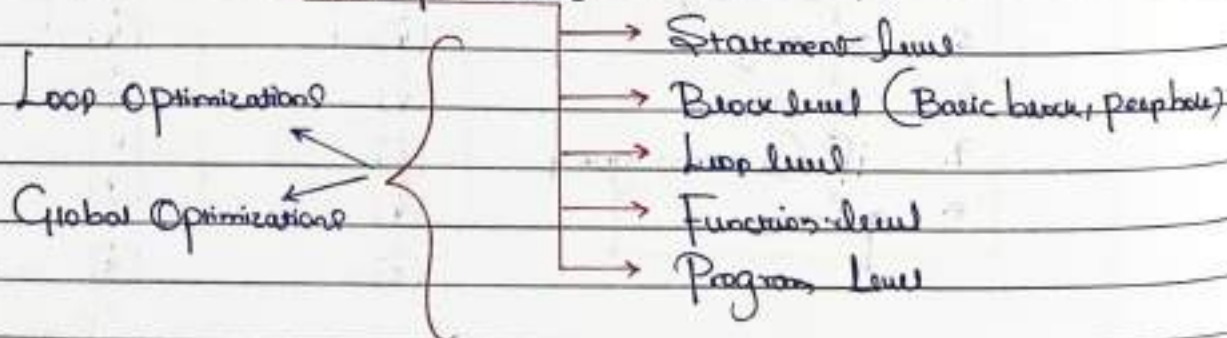
No of nodes = 6

No of edges = 6

Q1. Find equivalent expressions for the following syntax tree.



Ans $(a + (b * c)) * ((b * c) - a)$

Code OptimizationCode Optimization

Code Optimization Techniques:

1. Constant folding
2. Copy Propagation
 - Constant propagation
 - Variable propagation
3. Common Sub Expression Elimination
4. Strength reduction
5. Algebraic Simplifications
6. Dead code elimination
7. Loop Optimizations
 - Code motion
 - induction variable elimination
 - loop merging
 - loop unrolling

① Constant Folding:

$$x = 2 * 3 + y$$

constant folding

$$x = 6 + y$$

② Copy Propagation

I.

$$x = 20$$

$$y = x + a$$

constant propagation
replace x with 20

$$x = 20$$

$$y = 20 + a$$

dead code elimination

$$y = 20 + a$$

II.

$$x = b$$

$$y = x + a$$

variable propagation
replace x with 'b'

$$x = b$$

$$y = b + a$$

dead code elimination

$$y = b + a$$

③ Common Sub-Expression Elimination

→ DAG can be used

$$x = (a+b) * (a+b) \Rightarrow t_1 = a+b$$

$$x = t_1 * t_1$$

④ Strength Reduction:

It replaces Costlier code with cheaper.

$$\underbrace{x = a * 2}_{\substack{\text{multiplication} \\ \text{is} \\ \text{Costlier}}} \longrightarrow \left. \begin{array}{l} x = a + a \\ x = a \ll 1 \end{array} \right\} \text{Cheaper}$$

⑤ Algebraic Simplifications:

Cancellation law (i) $x = a + \cancel{b} - \cancel{b} + c \longrightarrow x = a + c$

Identity law (ii) $x = a + \underline{b} * 1 \longrightarrow x = a + b$

Identity law (iii) $x = a + \underline{b} + 0 \longrightarrow x = a + b$

Domination (iv) $x = a + \underline{b} + c * 0 \longrightarrow x = a + b$

⑥ Dead Code Elimination:

$$\boxed{x = a + b} - \text{dead code (useless)}$$

$$y = a + b$$

$$z = y + c$$

$$\text{Print}(z)$$



$$y = a + b$$

$$z = y + c$$

$$\text{Print}(z)$$

⑦ Loop Optimizations:(i) Code Motion:

```
for (i = 0; i < n; i++)
{
    x = a + b;
    y = x * i;
}
```

identify
loop invariant code

and move
outside
loop

```
x = a + b
for (i = 0; i < n; i++)
{
    y = x * i;
}
```

(ii) Induction Variables Elimination:

```

k=0;
for (i=0; j=0; i<n; i++)
{
  x = a+i;
  y = b*j;
  z = C+k;
  p = x*y+z;
  j = j+1;
  k = k+1;
}

```

⇒

```

for (i=0; i<n; i++)
{
  x = a+i;
  y = b*i;
  z = C*i;
  p = x*y+z;
}

```

(iii) Loop Merge

```

for (i=0; i<n; i++)
{
  A[i] = i+a;
  for (j=0; j<n; j++)
  {
    B[j] = j*b;
  }
}

```

⇒

```

for (i=0; i<n; i++)
{
  A[i] = i+a;
  B[i] = i*b;
}

```

(iv) Loop Unrolling:

half

```

for (i=0; i<4n; i++)
{
  printf("GATE");
}

```

⇒

```

for (i=0; i<2n; i++)
{
  printf("GATE");
  printf("GATE");
}

```

double

⇒

```

for (i=0; i<n; i++)
{
  printf("GATE");
  printf("GATE");
  printf("GATE");
  printf("GATE");
}

```

Data Flow Analysis

- To analyse programs
- To understand each variable
- Lattices can be used to analyse data
- Control flow graph can help to analyse data

Data Flow Analysis

Forward Analysis

Start

Example: reaching definitions
available expression

End analysis

Backward Analysis

Start of program

Example: live variables
analysis

End of program

Live Variable Analysis [Liveness Analysis]

- What is Live variable?
- How to compute live variables at any Statement?
- Backward Analysis:
 - Compute GEN(USE) & KILL(DEF) for each basic block
 - Compute In and Out for each BB.

Live Variable: x is live variable at Statement S_i if:

- I. There exists Statement S_j that reads x
- II. There exists path from S_i to S_j
- III. There is no assignment into x before S_j .

Find live variables at Statement 1, 2, 3, & 4

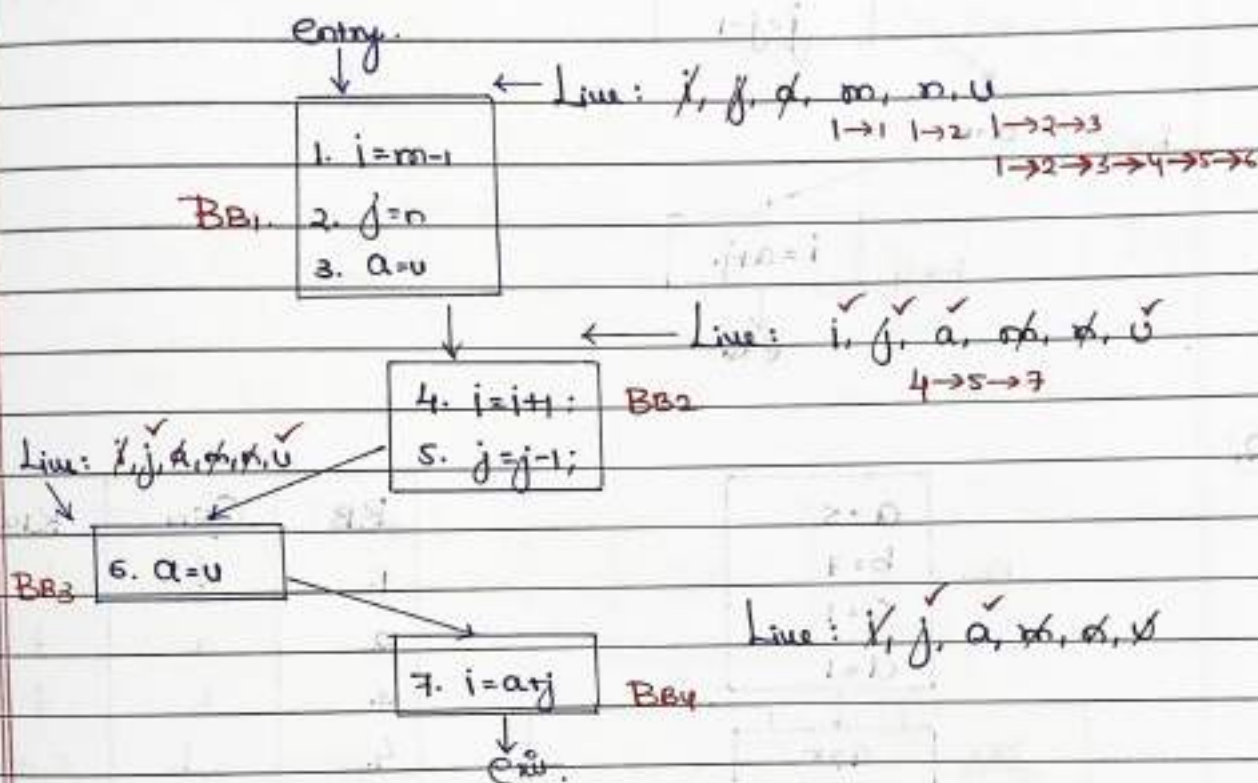
1.	$x = a + b;$	$\checkmark, \checkmark, a, b, c$ 1 → 1 1 → 1 1 → 2 1 → 2 → 3 → 4	4.	$b = a - c$	$\checkmark, \checkmark, a, \checkmark, c$
2.	$y = x * c;$	$\checkmark, \checkmark, \checkmark, \checkmark, c$ 2 → 2 2 → 2 2 → 3 2 → 3 → 4			
3.	$a = x + y;$	$\checkmark, \checkmark, \checkmark, \checkmark, c$ 3 → 3 3 → 3 3 → 4			

1. $a = a + b \leftarrow A+1: a, b, \phi$
 $1 \rightarrow 1 \quad 1 \rightarrow 1$

2. $b = a + b \leftarrow A+2: a, b, \phi$
 $2 \rightarrow 2 \quad 2 \rightarrow 2$
 $2 \rightarrow 3$

3. $c = b + a \leftarrow A+3: a, b, \phi$
 $3 \rightarrow 3 \quad 3 \rightarrow 3$

eg:

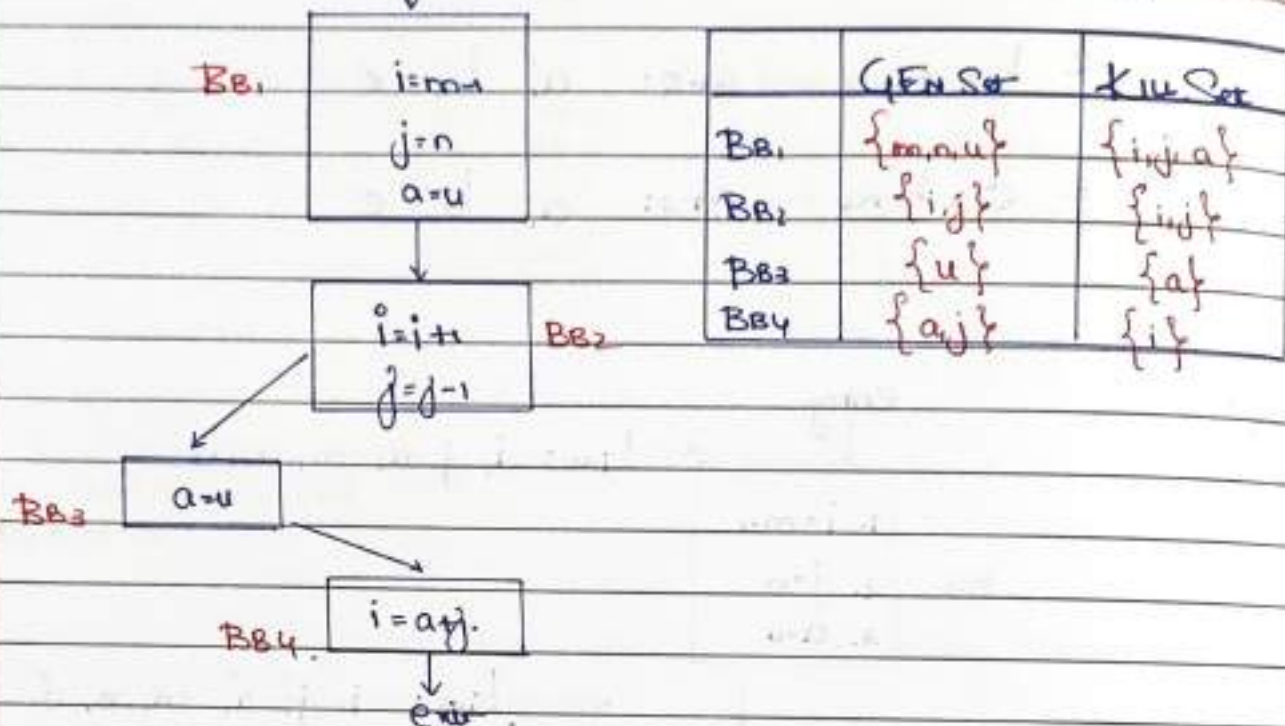


GEN (USE) Set	KILL (DEF) Set
BB_k $x = a + b$ $y = x * c$ $z = y - d$ $GEN_k = \{a, b, c, d\}$ $= \{v \mid v \text{ is used at some statement in } BB_k \text{ and no definition before the statement}\}$	$KILL_k = \{x, y, z\}$ $= \{v \mid v \text{ has definition in } BB_k\}$

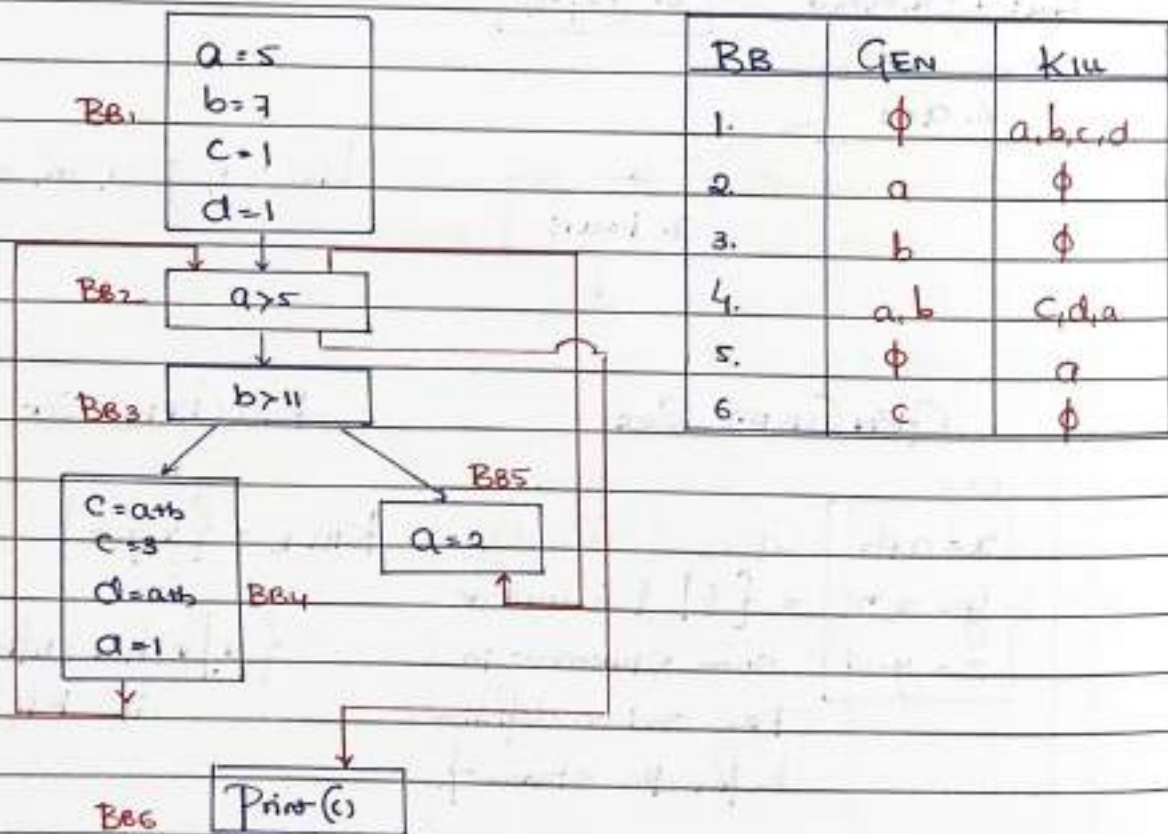
①

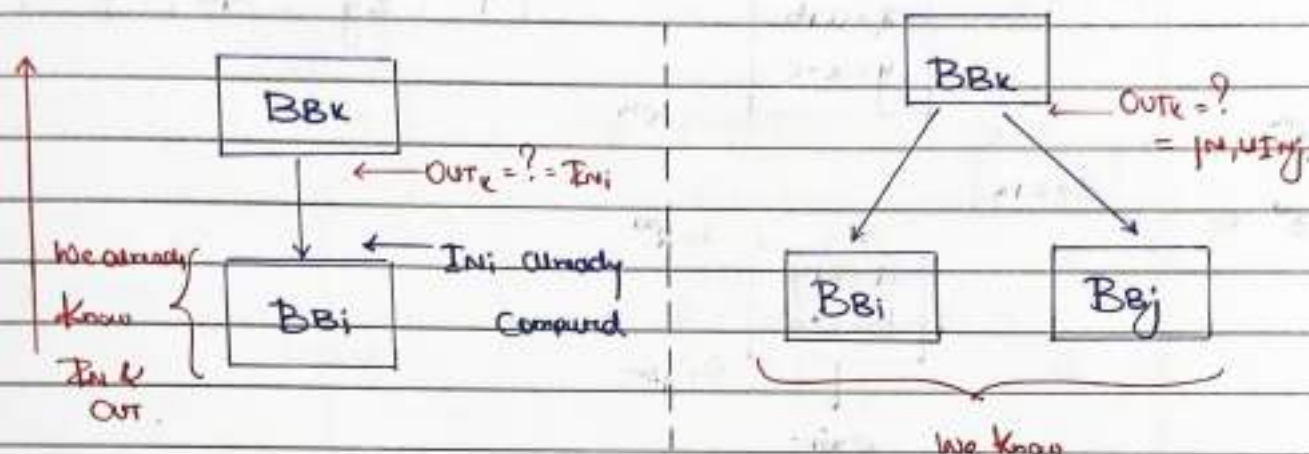
entry

Compute GEN & Kill for each BB

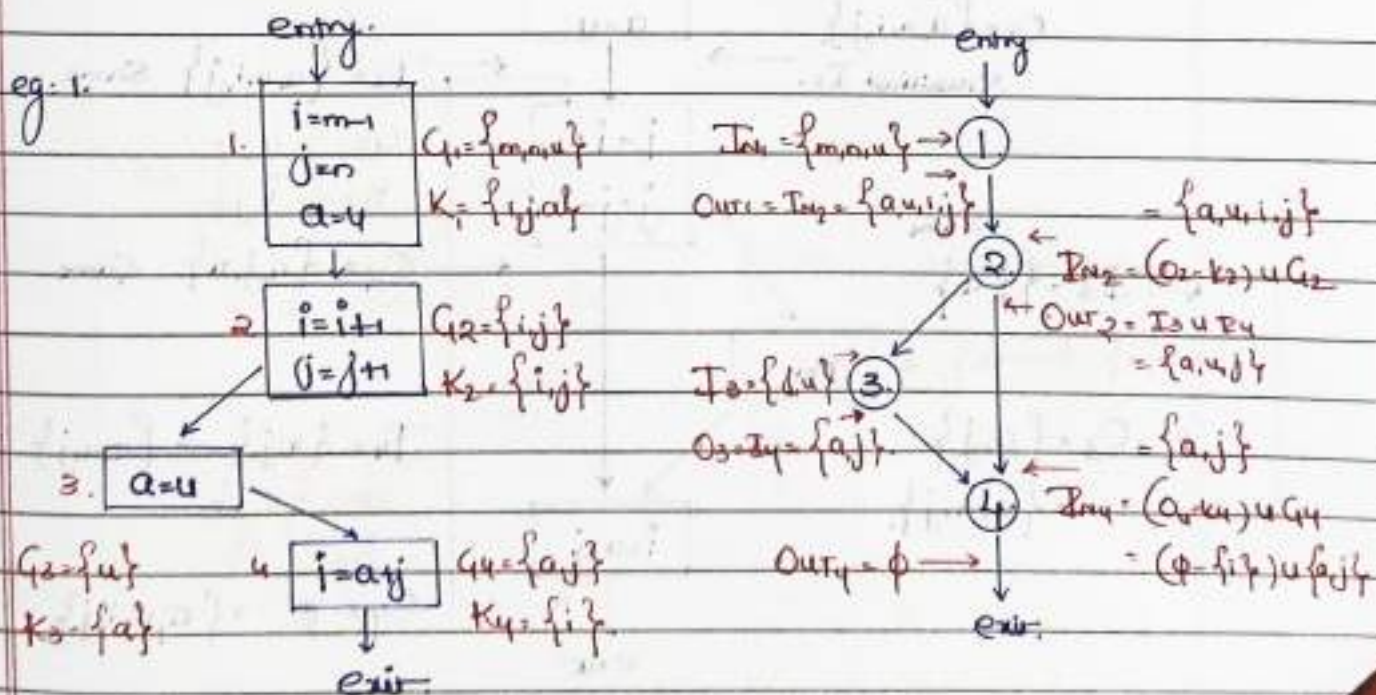
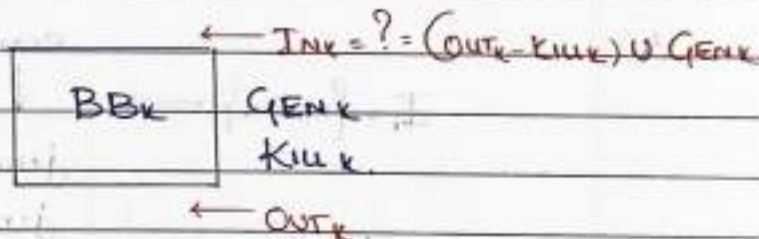


②

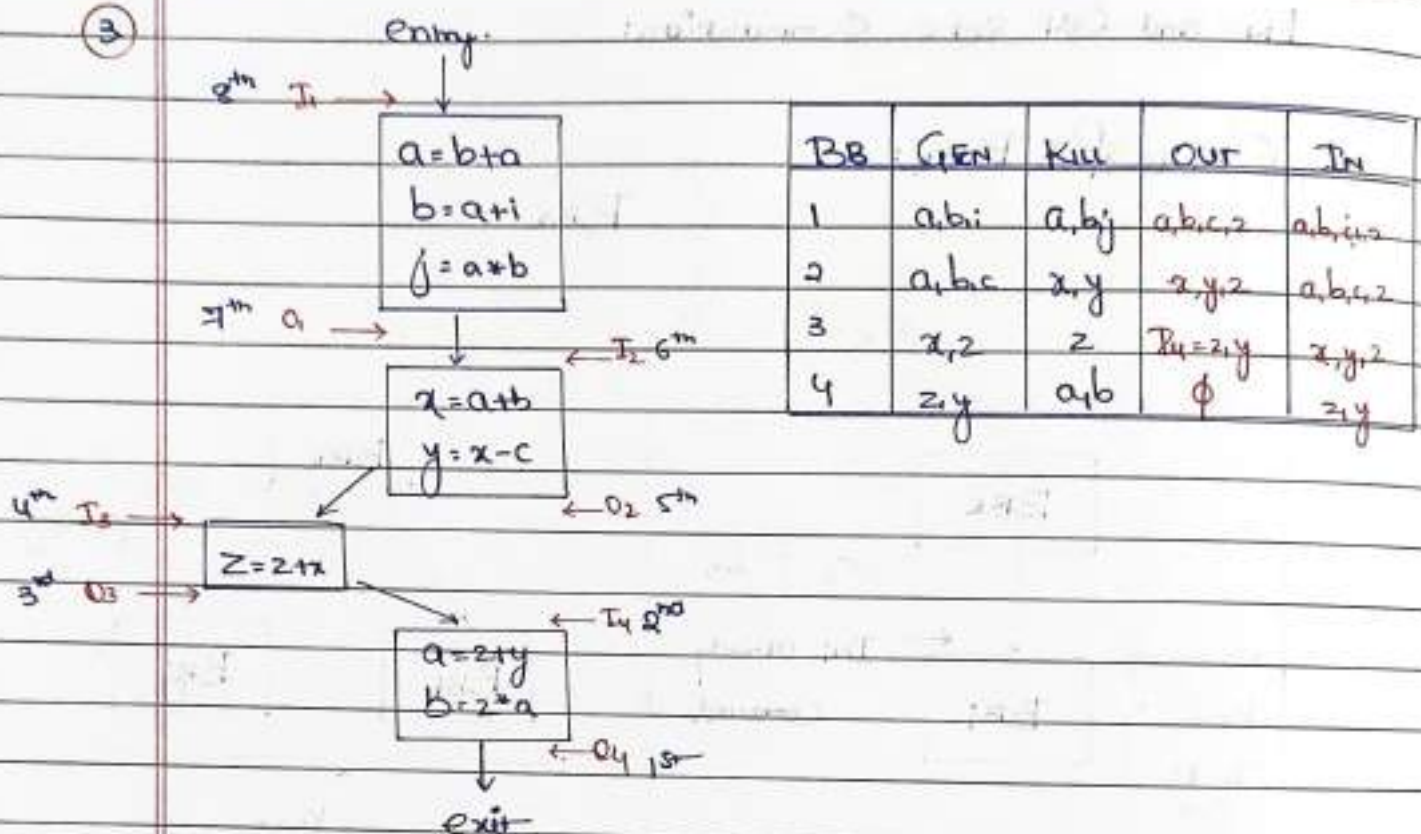


IN and Out Sets Computation:OUT_k for BB_k:OUT_k = ? →IN_k for BB_k:

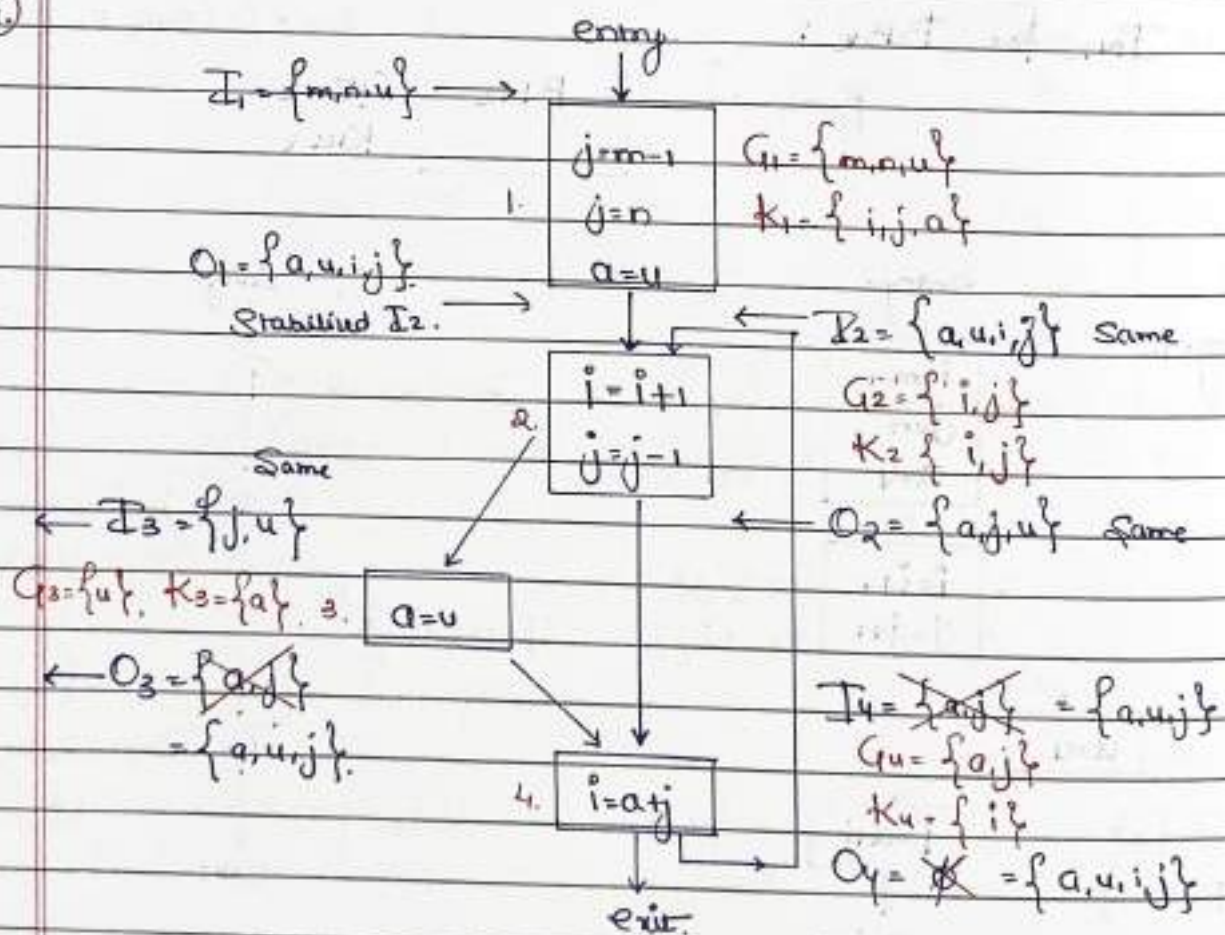
Backward ↑

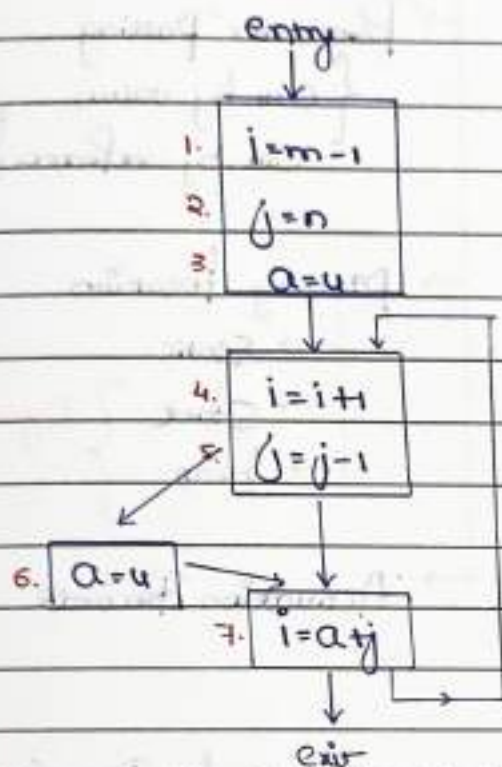


③



④



Reaching DefinitionsI USE-DEF Chain

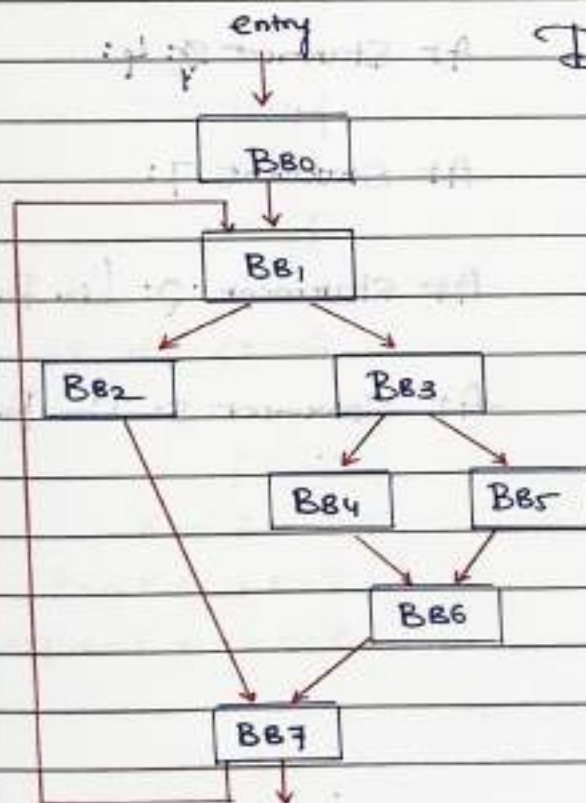
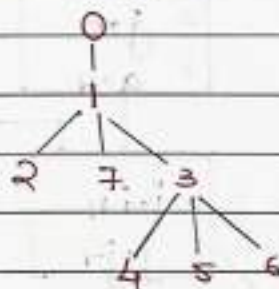
At statement 5: $5 \rightarrow 2$
 $5 \rightarrow 5$

At statement 7: $7 \rightarrow 5$

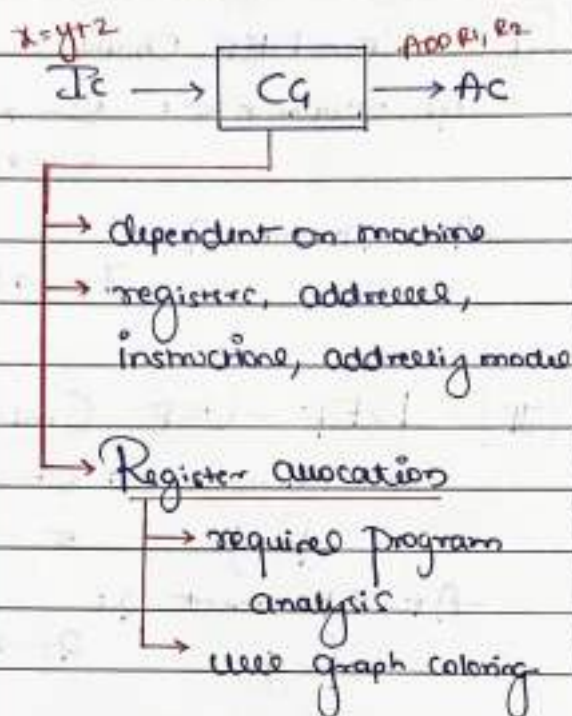
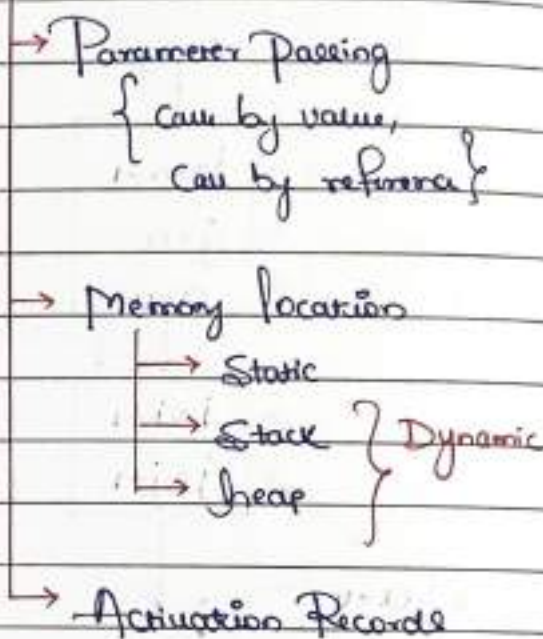
II DEF-USE Chain

At statement 5: $5 \rightarrow 7$
 for j $5 \rightarrow 5$

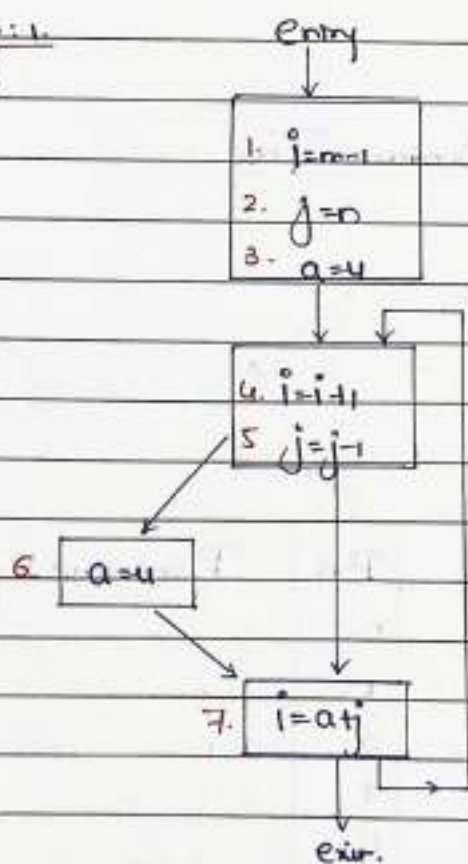
At statement 2: $2 \rightarrow 5$
 for j

Dominator Tree

BB	Dominated By
0	0
1	0, 1
2	0, 1, 2
3	0, 1, 3
4	0, 1, 3, 4
5	0, 1, 3, 5
6	0, 1, 3, 6
7	0, 1, 7

Code GenerationRun Time Environment

eg: 1.

At Statement 1: $\text{Lin Range}(i) = ?$ $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

At Statement 4:

Null

At Statement 7:

 $7 \rightarrow 4$ At Statement 2: $\text{Lin Range}(j) = ?$ $2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ At Statement 5: $\text{Lin Range}(j) = ?$ $5 \rightarrow 7$ $5 \rightarrow 6 \rightarrow 7$ $5 \rightarrow 7 \rightarrow 4 \rightarrow 5$ $5 \rightarrow 6 \rightarrow 7 \rightarrow 4 \rightarrow 5$ Note:Static Memory Allocation $\rightarrow$ doesn't support recursionStack memory Allocation $\rightarrow$ supports recursion